The One Bitcoin Heist

Making a custom Hashcat plugin to solve a decade-old puzzle challenge

By Joseph Gabay

v3

DEFCON TALK DISCLAIMER
WARNING

This talk reflects my personal research project and opinions, not my employer’s or anyone else’s.

Any views, opinions, or research presented in this talk are personal and belong solely to me. They do not represent or reflect those of any person, institution, or organization that I may or may not be associated with in a professional or personal capacity unless explicitly stated otherwise. All other opinions generated by CSRNG. Not ANSI Compliant. All facts presented in this presentation are certified to be true and factual, unless they aren’t. No LLMs were harmed in the making of this presentation. No warranty is expressed, implied, asserted, regifted or refried. Some crimes may or may not be illegal, and punishable by fines of up to or over $250,000.

Please spay and neuter your pets.

Thank you.

What this talk is about

In short:

- Over 10 Years Ago (2014) some anonymous person posted a puzzle challenge with a 1BTC Prize - first to solve 20 clues unlocked the brainwallet containing the crypto
- It went unclaimed for a long, long time.
- I got close, but didn't have all 20 solutions.
- I ended up writing a plugin for Hashcat to crack brainwallet, and brute forced it

This talk will talk about:

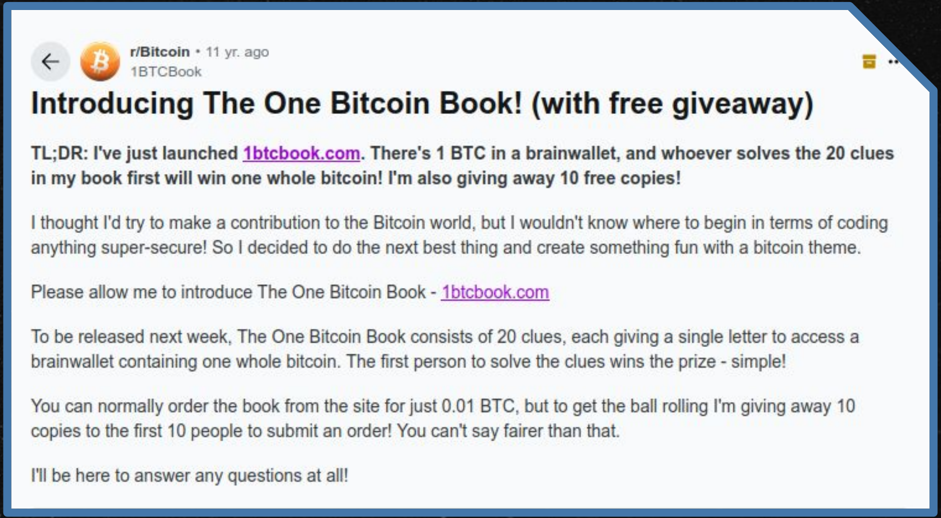
- The puzzle itself, who made it, and how it worked
- How brainwallets work under the hood
- The basics of a hashcat module, and how to put them together

Who I am and what I do

- Robotics Engineer
- Cybersecurity Researcher
- Physical and Embedded Pentester
- Skydiver
- Shopping Cart Phreaker
- DC29 & DC31 Talks
- Flat Moon Conspiracist
- Internet Treasure Hunter (Successful, Retired)

Introducing: *The One Bitcoin Book*

- Bitcoin Puzzle Challenge
  - Advertised around **April 2014**
  - First 10 to sign up were free
  - Others pay 0.01 BTC (~$5 at the time)
- Released by “Spencer Lucas”
  - No real identify I was able to find
  - You out there, man?
- 20 Clues, solve them all win 1 BTC
  - ~$500 at the time
  - Combine clues into “brainwallet” phrase
  - Also need secret string shared after purchase
- Prize address was public, could see status of the prize
  - 155LdnsyybcwCSzP77bATgSAn3KUF7nk3A



Introducing The One Bitcoin Book! (with free giveaway)

TL;DR: I've just launched 1btcbook.com. There's 1 BTC in a brainwallet, and whoever solves the 20 clues in my book first will win one whole bitcoin! I'm also giving away 10 free copies!

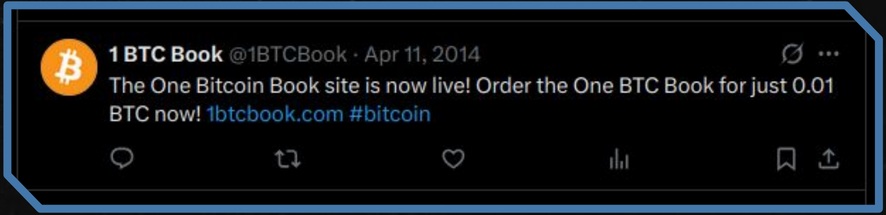
I thought I'd try to make a contribution to the Bitcoin world, but I wouldn't know where to begin in terms of coding anything super-secure! So I decided to do the next best thing and create something fun with a bitcoin theme.

Please allow me to introduce The One Bitcoin Book - 1btcbook.com

To be released next week, The One Bitcoin Book consists of 20 clues, each giving a single letter to access a brainwallet containing one whole bitcoin. The first person to solve the clues wins the prize - simple!

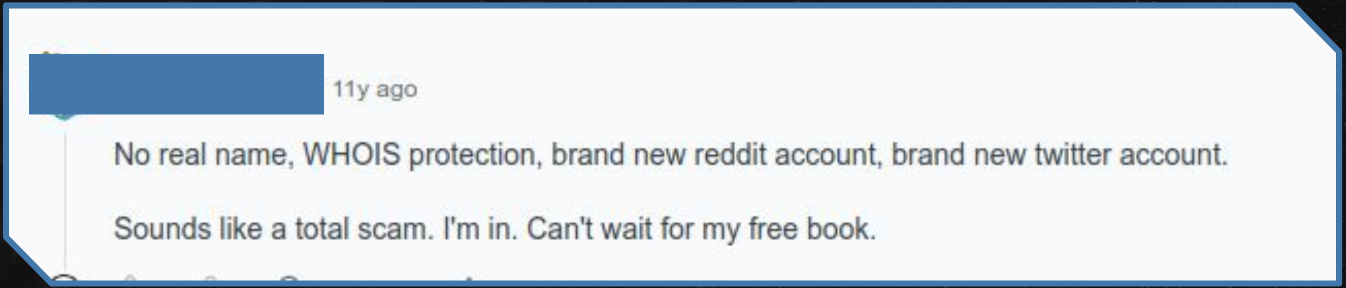
You can normally order the book from the site for just 0.01 BTC, but to get the ball rolling I'm giving away 10 copies to the first 10 people to submit an order! You can't say fairer than that.

I'll be here to answer any questions at all!



The One Bitcoin Book site is now live! Order the One BTC Book for just 0.01 BTC now! 1btcbook.com #bitcoin

Reactions were… mixed.



[redacted] 11y ago

No real name, WHOIS protection, brand new reddit account, brand new twitter account.

Sounds like a total scam. I'm in. Can't wait for my free book.

1BTCBook **OP** • 11y ago

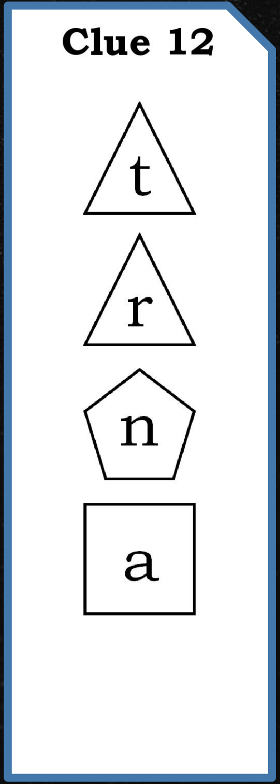
Valid concerns! I'm not sure how best to address them, other than it would be a ridiculously drawn-out scam just to get $4 from people!

This is more of a fun little side project that I'm hoping will gain some popularity, which I can then scale up to a 10 Bitcoin Book or similar.

Examples of clues, and solutions!

Clue 12 (solve credit to Ashley!):

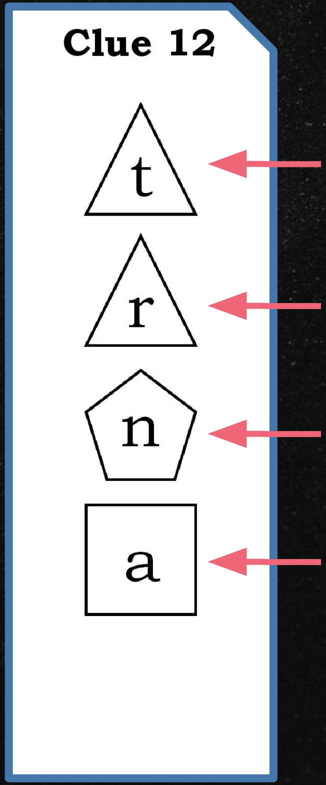
- Shapes with letters in them
- Find the pattern, predict the next letter



Examples of clues, and solutions!

Clue 12 (solve credit to Ashley!):

- Shapes with letters in them
- Find the pattern, predict the next letter
- Note each position index (n) and the name of the shape



Pos: ‘one’

Pos: ‘two’

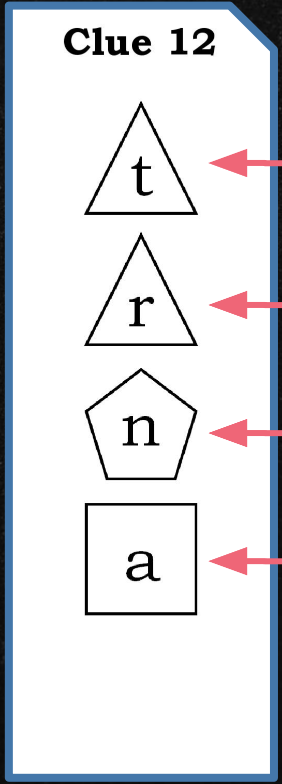
Pos: ‘three’

Pos: ‘four’

Examples of clues, and solutions!

Clue 12 (solve credit to Ashley!):

- Shapes with letters in them
- Find the pattern, predict the next letter
- Note each position index (n) and the name of the shape
- Shape has same number of sides as the position has letters
- Letter is n^{th} letter of shape name



Pos: ‘one’, shape: ‘triangle’

Pos: ‘two’, shape: ‘triangle’

Pos: ‘three’, shape: ‘pentagon’

Pos: ‘four’, shape: ‘square’

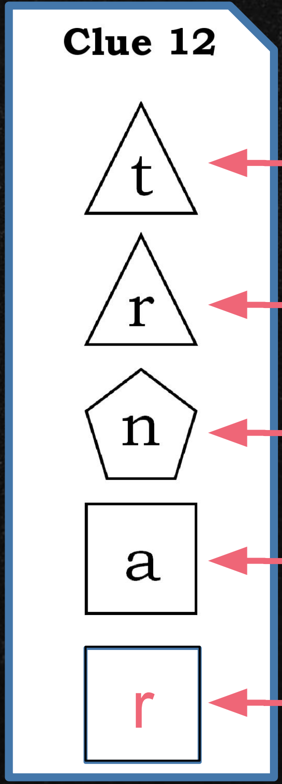
Examples of clues, and solutions!

Clue 12 (solve credit to Ashley!):

- Shapes with letters in them
- Find the pattern, predict the next letter
- Note each position index (n) and the name of the shape
- Shape has same number of sides as the position has letters
- Letter is n^{th} letter of shape name

5th letter

square



Pos: ‘one’, shape: ‘triangle’

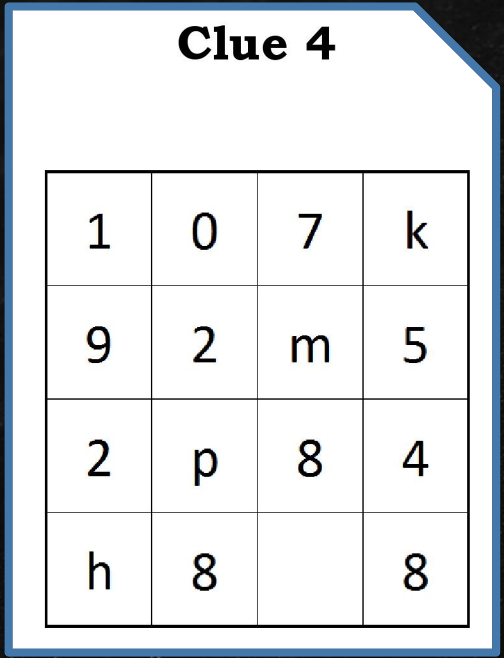
Pos: ‘two’, shape: ‘triangle’

Pos: ‘three’, shape: ‘pentagon’

Pos: ‘four’, shape: ‘square’

Pos: ‘five’, shape: ‘square’

Examples of clues, and solutions, cont.



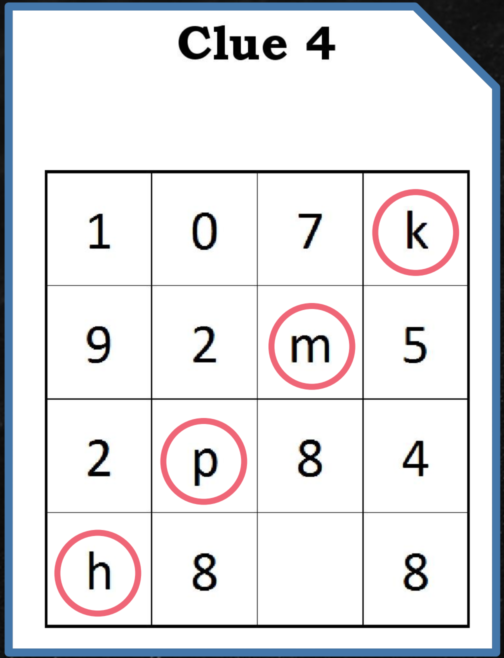
Clue 4

1	0	7	k
9	2	m	5
2	p	8	4
h	8		8

Clue 4:

- 4x4 Grid with numbers and letters
- One empty space

Examples of clues, and solutions, cont.



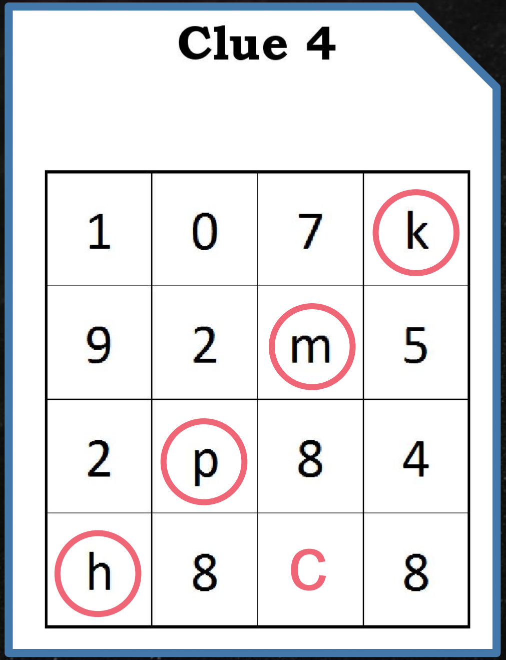
Clue 4

1	0	7	k
9	2	m	5
2	p	8	4
h	8		8

Clue 4:

- 4x4 Grid with numbers and letters
- One empty space
- Can see “kmph” in diagonal
- Other numbers are “1077252848_8”

Examples of clues, and solutions, cont.



Clue 4

1	0	7	k
9	2	m	5
2	p	8	4
h	8	c	8

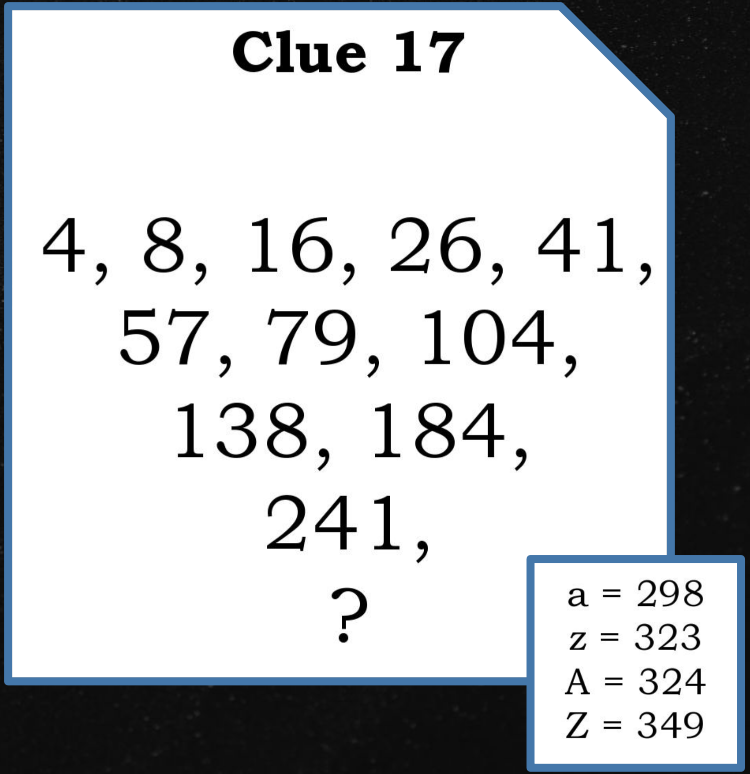
Clue 4:

- 4x4 Grid with numbers and letters
- One empty space
- Can see “kmph” in diagonal
- Other numbers are “1077252848_8”
- Speed of light in kmph: 1077252848.8
- Answer is ‘**c**’, constant for speed of light

Examples of clues, and solutions, cont.

Clue 17: (Another Ashley solve!)

- Sequence of 12 numbers in 5 rows
- Last number is ?, key to decode letter



Clue 17

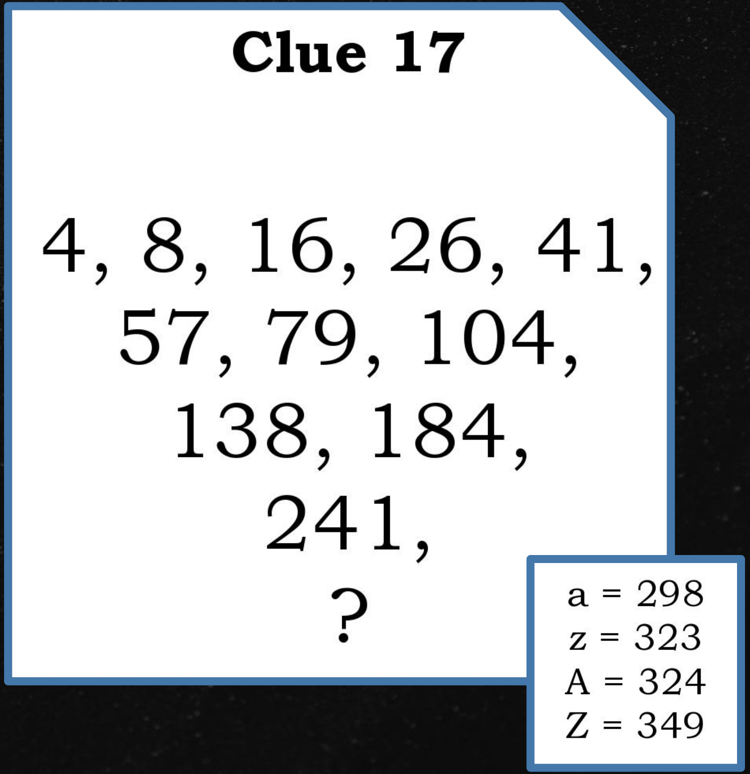
4, 8, 16, 26, 41,
57, 79, 104,
138, 184,
241,
?

a = 298
z = 323
A = 324
Z = 349

Examples of clues, and solutions, cont.

Clue 17: (Another Ashley solve!)

- Sequence of 12 numbers in 5 rows
- Last number is ?, key to decode letter
- N^{th} number is sum of N^{th}:
    - Prime Number (2, 3, 5, 7, 11, 13...)
    - Fibonacci Number (1, 1, 2, 3, 5, 8...)
    - Perfect Square (1, 4, 9, 16, 25, 36...)



Clue 17

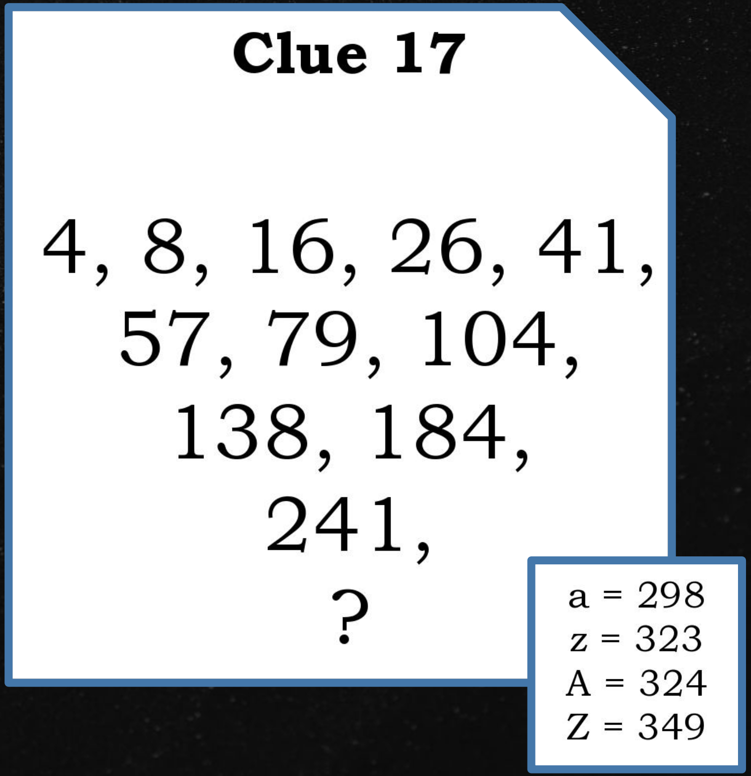
4, 8, 16, 26, 41,
57, 79, 104,
138, 184,
241,
?

a = 298
z = 323
A = 324
Z = 349

Examples of clues, and solutions, cont.

Clue 17: (Another Ashley solve!)

- Sequence of 12 numbers in 5 rows
- Last number is ?, key to decode letter
- Nth number is sum of Nth:
  - Prime Number (2, 3, 5, 7, 11, 13…)
  - Fibonacci Number (1, 1, 2, 3, 5, 8…)
  - Perfect Square (1, 4, 9, 16, 25, 36…)
- 12th is: 37 + 144 + 144 = **325** = **B**



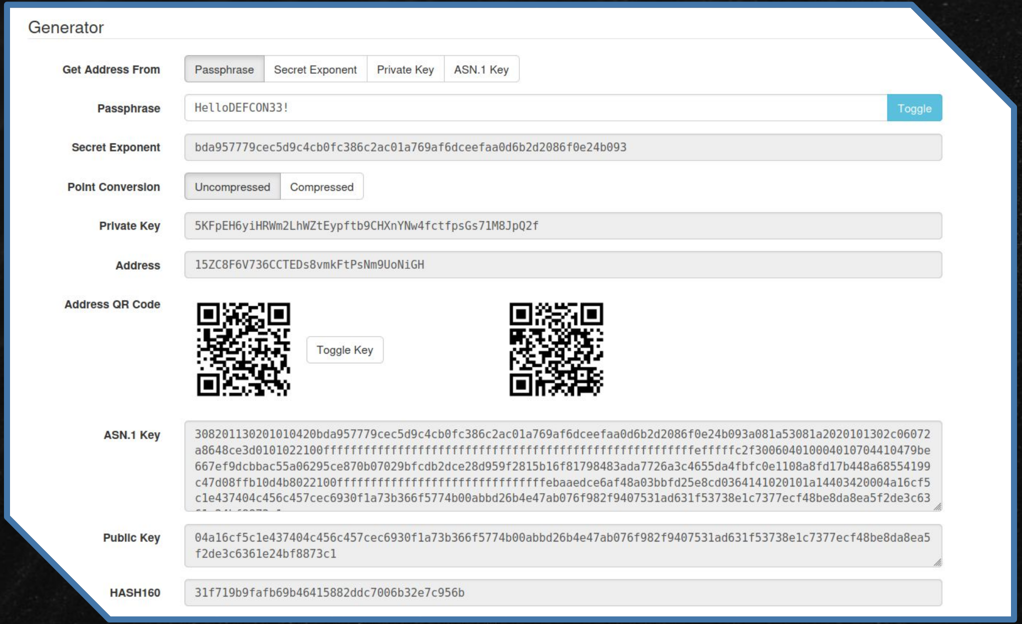
Clue 17

4, 8, 16, 26, 41,
57, 79, 104,
138, 184,
241,
?

a = 298
z = 323
A = 324
Z = 349

Okay, what *is* a brainwallet?

**** WARNING DO NOT USE THESE ****



Generator

Get Address From	Passphrase | Secret Exponent | Private Key | ASN.1 Key
Passphrase	HelloDEFCON33!
Secret Exponent	bda957779cec5d9c4cb0fc386c2ac01a769af6dceefaa0d6b2d2086f0e24b093
Point Conversion	Uncompressed | Compressed
Private Key	5KFpEH6yiHRWm2LhWZtEypftb9CHXnYNw4fctfpsGs71M8JpQ2f
Address	15ZC8F6V736CCTEDs8vmkFtPsNm9UoNiGH
Address QR Code	
ASN.1 Key	308201130201010420bda957779cec5d9c4cb0fc386c2ac01a769af6dceefaa0d6b2d2086f0e24b093a081a53081a2020101302c06072 a8648ce3d0101022100fffffffffffffffffffffffffffffffffffffffffffffffffffffffefffffc2f300604010004010704410479be 667ef9dcbbac55a06295ce870b07029bfcdb2dce28d959f2815b16f81798483ada7726a3c4655da4fbfc0e1108a8fd17b448a68554199 c47d08ffb10d4b8022100fffffffffffffffffffffffffffffffebaaedce6af48a03bbfd25e8cd0364141020101a14403420004a16cf5 c1e437404c456c457cec6930f1a73b366f5774b00abbd26b4e47ab076f982f9407531ad631f53738e1c7377ecf48be8da8ea5f2de3c63
Public Key	04a16cf5c1e437404c456c457cec6930f1a73b366f5774b00abbd26b4e47ab076f982f9407531ad631f53738e1c7377ecf48be8da8ea5 f2de3c6361e24bf8873c1
HASH160	31f719b9fafb69b46415882ddc7006b32e7c956b

- Store your bitcoin in your brain
- Turns secret passphrase into bitcoin address
- No longer considered secure
  - More on this in a sec
- Use your secret passphrase to calculate your private key
- brainwalletx.github.io
  - Probably don't trust this with anything important

Brief Intro to Cryptography: Hashes

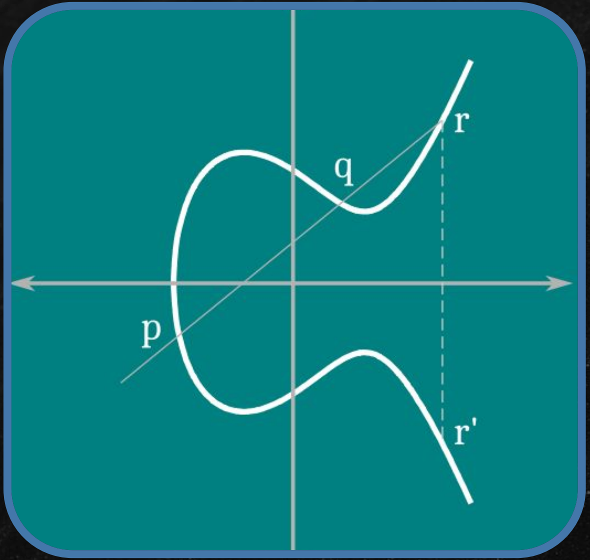
Hash: a function that converts an input of arbitrary length to a fixed-sized output, in manner that is both **one-way** and **repeatable**. That is:

- Hashing the same string twice gives same result
- You cannot (easily) find original input from an output
- The output is always the same size
    - E.g, SHA-256 has a 256 byte output

They are used in:

- Storing passwords/credentials
- Verifying data integrity
- Wasting time in cryptography (on purpose)

Brief Intro to Cryptography: Elliptic Curve Cryptography



Elliptic Curve Cryptography - Using the properties of special algebraic curves to do cryptography

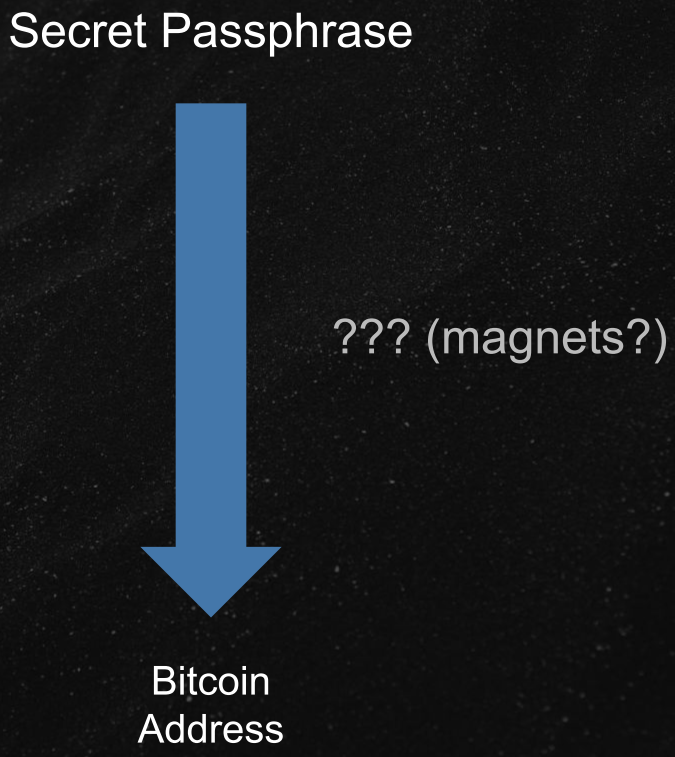
- Uses 2 keys: public key and private key
- You can **derive** public key from private key
- Cannot derive private key from public key
- Private key **signs**, public key **verifies**

Used in:

- Secure communication
- Cryptocurrencies
- Authentication/Identity verification

How do Brainwallets work?

How do Brainwallets work?

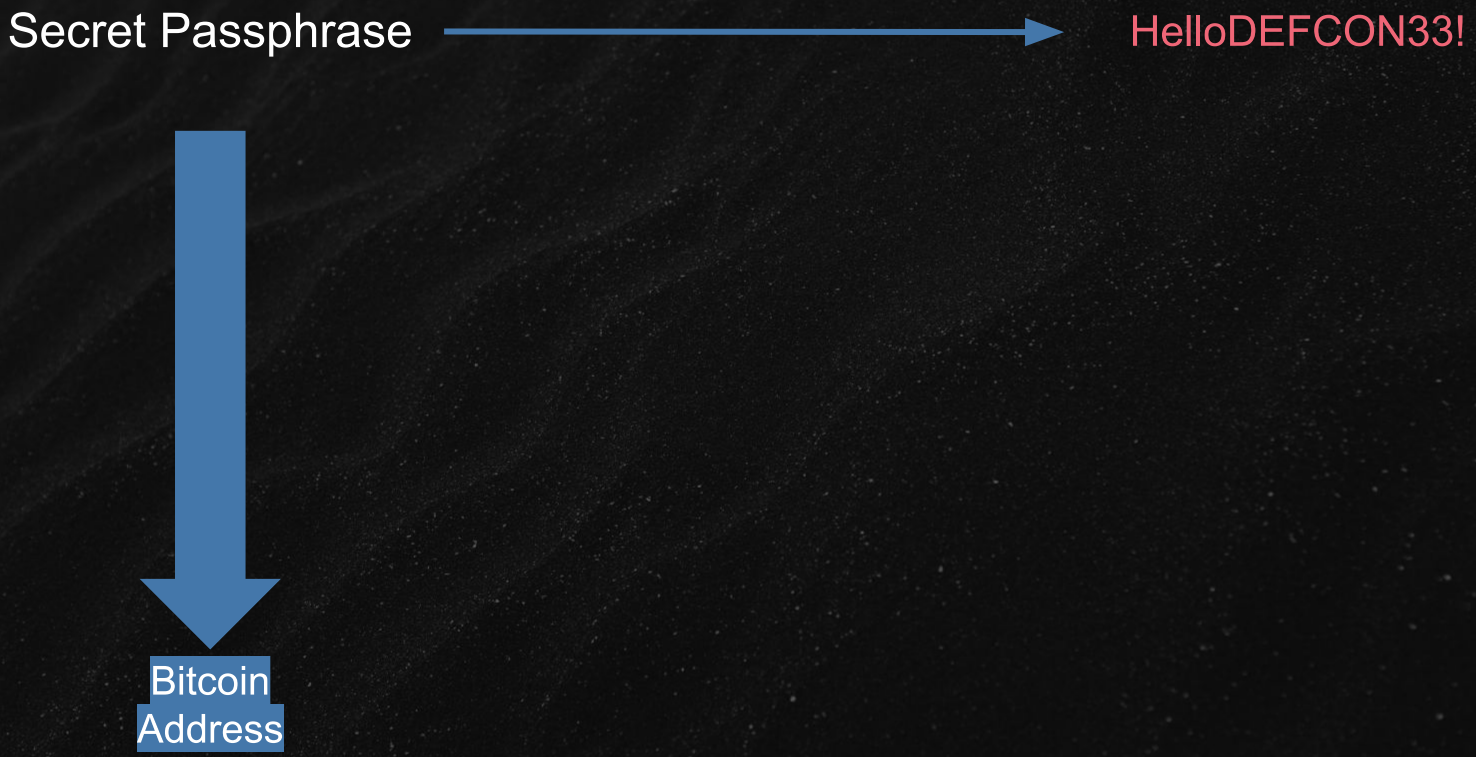


Secret Passphrase

??? (magnets?)

Bitcoin
Address

How do Brainwallets work?

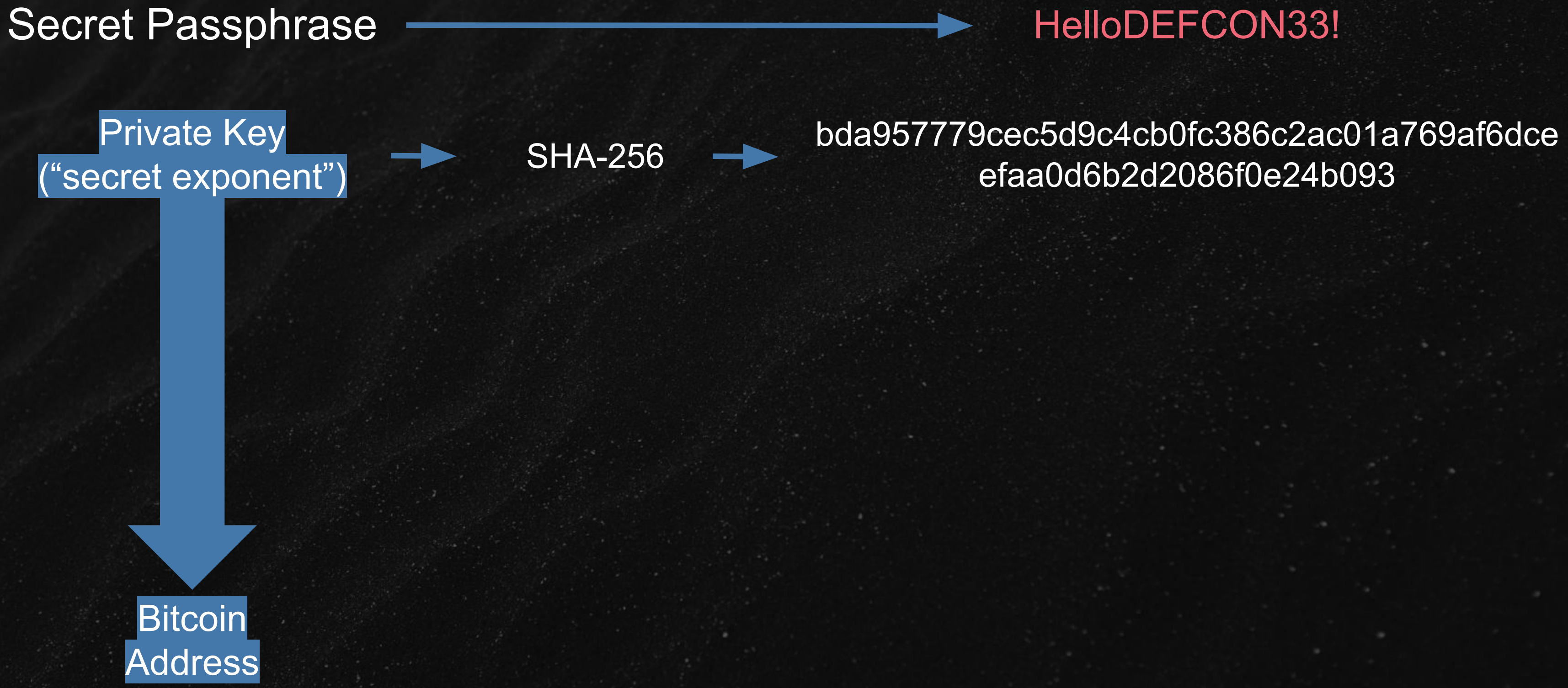


Secret Passphrase

HelloDEFCON33!

Bitcoin Address

How do Brainwallets work?

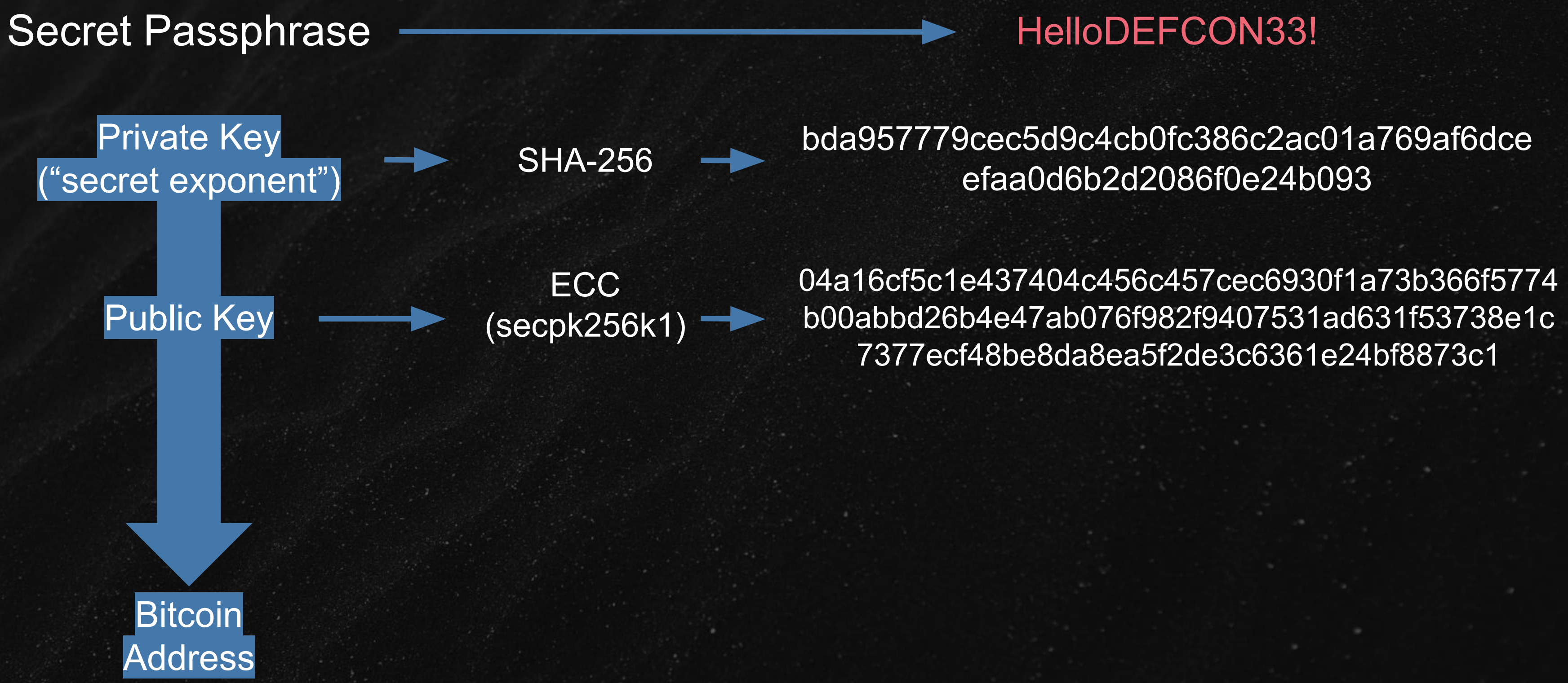


Secret Passphrase → HelloDEFCON33!

Private Key
("secret exponent") → SHA-256 → bda957779cec5d9c4cb0fc386c2ac01a769af6dce
efaa0d6b2d2086f0e24b093

Bitcoin
Address

How do Brainwallets work?



Secret Passphrase → HelloDEFCON33!

Private Key
(“secret exponent”) → SHA-256 → bda957779cec5d9c4cb0fc386c2ac01a769af6dce
efaa0d6b2d2086f0e24b093

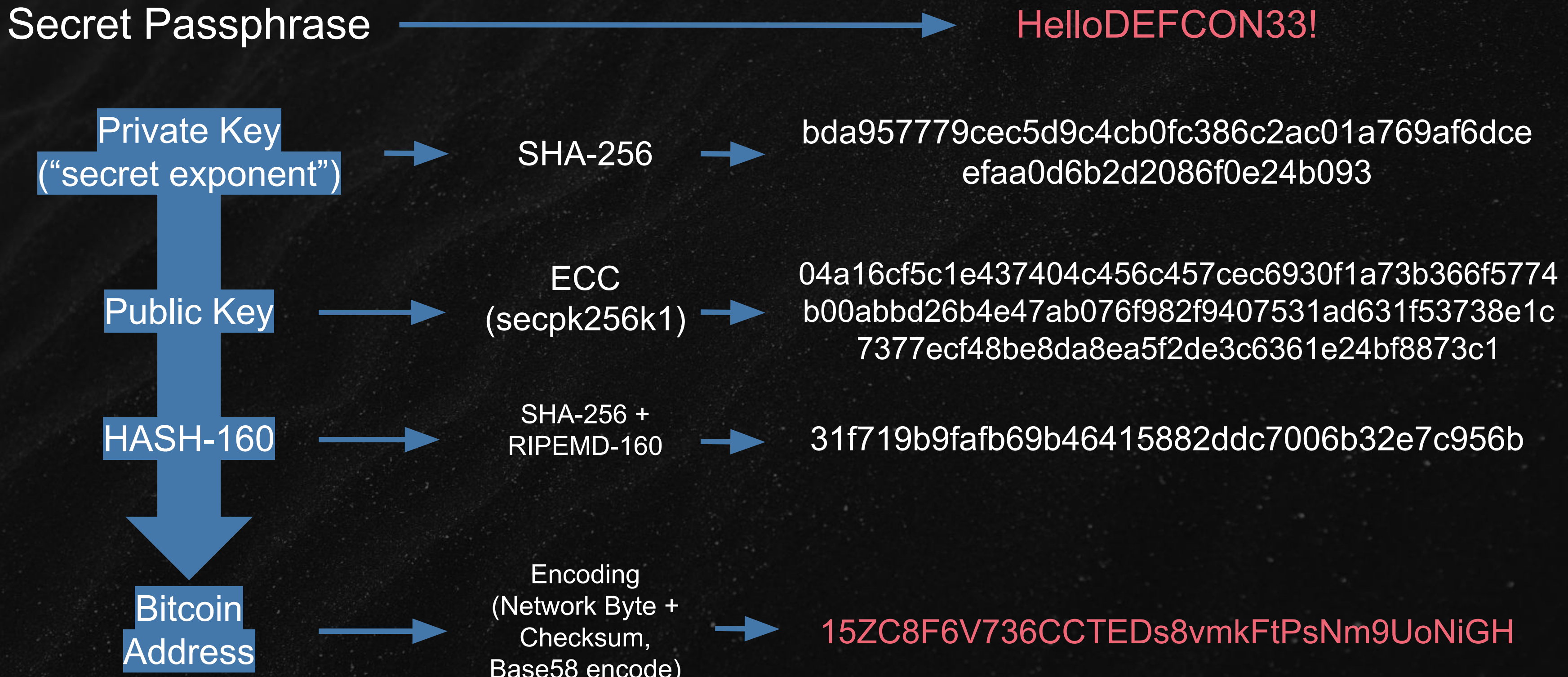
Public Key → ECC
(secpk256k1) → 04a16cf5c1e437404c456c457cec6930f1a73b366f5774
b00abbd26b4e47ab076f982f9407531ad631f53738e1c
7377ecf48be8da8ea5f2de3c6361e24bf8873c1

Bitcoin
Address

How do Brainwallets work?

Secret Passphrase		HelloDEFCON33!
Private Key (“secret exponent”)	SHA-256	bda957779cec5d9c4cb0fc386c2ac01a769af6dce efaa0d6b2d2086f0e24b093
Public Key	ECC (secpk256k1)	04a16cf5c1e437404c456c457cec6930f1a73b366f5774 b00abbd26b4e47ab076f982f9407531ad631f53738e1c 7377ecf48be8da8ea5f2de3c6361e24bf8873c1
HASH-160	SHA-256 + RIPEMD-160	31f719b9fafb69b46415882ddc7006b32e7c956b
Bitcoin Address		

How do Brainwallets work?



Secret Passphrase → HelloDEFCON33!

Private Key ("secret exponent")	SHA-256	bda957779cec5d9c4cb0fc386c2ac01a769af6dce efaa0d6b2d2086f0e24b093
Public Key	ECC (secpk256k1)	04a16cf5c1e437404c456c457cec6930f1a73b366f5774 b00abbd26b4e47ab076f982f9407531ad631f53738e1c 7377ecf48be8da8ea5f2de3c6361e24bf8873c1
HASH-160	SHA-256 + RIPEMD-160	31f719b9fafb69b46415882ddc7006b32e7c956b
Bitcoin Address	Encoding (Network Byte + Checksum, Base58 encode)	15ZC8F6V736CCTEDs8vmkFtPsNm9UoNiGH

Solving the puzzle the intended way

- Solve all 20 Clues
- Append “secret string” from the email
- Put into brainwallet website
  - Now brainwalletx after security flaws were revealed
  - Probably should do this offline
- Import private key, sweep address
- ???
- Profit!

Solving the puzzle the hacker way

- High confidence in ~70% of clues
- Decent guess for ~20%
- No idea on last ~10%
- **What if we just brute forced the last 30%?**

A1 fx n

	A	B	C	D
1	n		Answer	# Possible
2	1	1	b	1
3	2	2	Y	1
4	3	3	A	1
5	4	4	c	1
6	5	5	j	1
7	6	6	S/M	52
8	7	7	C	1
9	8	8	K	1
10	9	9	Q	1
11	10	10	i	52
12	11	11	F	52
13	12	12	r	1
14	13	13	I	52
15	14	14	N/E	2
16	15	15		52
17	16	16	t?	52
18	17	17	B	1
19	18	18		52
20	19	19	w	1
21	20	20	Y	1
22				
			total combos:	2,056,143,405,056

Solving the puzzle the hacker way

- High confidence in ~70% of clues
- Decent guess for ~20%
- No idea on last ~10%
- **What if we just brute forced the last 30%?**
- It's time to do some

MATH

A1 | fx n

	A	B	C	D
1	n		Answer	# Possible
2	1	1	b	1
3	2	2	Y	1
4	3	3	A	1
5	4	4	c	1
6	5	5	j	1
7	6	6	S/M	52
8	7	7	C	1
9	8	8	K	1
10	9	9	Q	1
11	10	10	i	52
12	11	11	F	52
13	12	12	r	1
14	13	13	I	52
15	14	14	N/E	2
16	15	15		52
17	16	16	t?	52
18	17	17	B	1
19	18	18		52
20	19	19	w	1
21	20	20	Y	1
22				
			total combos:	2,056,143,405,056

Lesson Time: Combinatorics

- **Combinatorics: the branch of math dealing with counting combinations of objects in a set**
- 20 positions, each having 52 possible values:
  - 52^{20} = 20896178655943101411324274803736576
  - Or: 2.09×10^{34}
  - That's a lotta zeroes
- Each solved clue divides this by 52
- We can figure out how many keys we need to test
- There are too many to test manually

Solved	Possibilities
12	5.35E+13
13	1.03E+12
14	1.98E+10
15	3.80E+08
16	7311616
17	140608
18	2704
19	52
20	1

Cracking Brainwallets - Brainflayer

- Utility for brute-forcing Brainwallets
- Written by Ryan Castellucci
  - Fellow nonbinary hacker, suing UK government for gender recognition
  - https://github.com/ryancdotorg/brainflayer
- Featured in DEF CON 23 talk
  - “Cracking CryptoCurrency Brainwallets”
- Is the reason you should not use Brainwallets
  - Or do whatever, I’m not your mom or accountant
- Runs only in CPU, no GPU support

I feel the need, the need for... hashrate.

- My home PC maxes around 1.4 MHash/sec
  - Intel(R) Core(TM) i7-6700K CPU @ 4.00GHz
  - 32 GB RAM

I feel the need, the need for... hashrate.

- My home PC maxes around 1.4 MHash/sec
  - Intel(R) Core(TM) i7-6700K CPU @ 4.00GHz
  - 32 GB RAM
- Now we get to start dividing big numbers!
  - 1.4 MHash = 1.4×10^6
  - 2.09×10^{34} hashes / 1.4×10^6 Mhash/S= 1.49×10^{28} seconds = **4.73×10^{20} years**
    - Age of universe is on order of 10^{10} years

I feel the need, the need for... hashrate.

- My home PC maxes around 1.4 MHash/sec
    - Intel(R) Core(TM) i7-6700K CPU @ 4.00GHz
    - 32 GB RAM
- Now we get to start dividing big numbers!
    - 1.4 MHash = 1.4×10^{6}
    - 2.09×10^{34} hashes / 1.4×10^{6} Mhash/S= 1.49×10^{28} seconds = **4.73×10^{20} years**
        - Age of universe is on order of 10^{10} years
- Golly-gee it’s good we solved some!
- 15 solved = 3.80×10^{08} hashes = **~271s**
    - That’s doable!

How long would it take to solve with N known clues?

Number of Known Clues	Hashes to Check	Solve time at 1.4 MHash/s
9	7.52E+18	170,000 years
10	1.45E+17	3275 years
11	2.78E+15	63 years
12	5.35E+13	442 days
13	1.03E+12	8.5 days
14	1.98E+10	4 hours
15	3.80E+08	272 seconds
16	7,311,616	5 seconds

Cracking the Puzzle

- Wrote a lightweight “generator” script
  - C program for speed
  - Output piped directly to Brainflayer
- Held 20 arrays, each with a character set
- A whole buncha `for` loops nested together
- Set up my known clues and the “extra string”
- Set all other clues to wildcard
- Let it rip through 19.8 billion hashes

FOUR HOURS LATER

Well, that was eas-

- Nope. Well crap.
- Even with 7 clues set to wildcard (over 8 days to run)
- Tried multiple combinations of known/unknown
- Something wasn’t right, but no idea where
    - No feedback in process
    - No idea where to look
- I need to try a different approach

More combinatorics? You said there was gonna be cats…

- We don’t know which “known” is incorrect
- We have to cycle through all of them as wildcards
- This increases the search space by a factor given by Pascal’s Rule

$$C(n,k) = \binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

- n is total number of known clues to pick from
- k is number of rotating wildcards
- **!** is factorial, not just my excitement to be here
- Solve space becomes $C*(52^{n+k})$

Increase Factor for Different k and n values

Rotating wildcards (k)

Fixed wildcards (20-n)

	1	**2**	**3**	**4**	**5**
1	19	171	969	3876	11628
2	18	153	816	3060	8568
3	17	136	680	2380	6188
4	16	120	560	1820	4368
5	15	105	455	1365	3003
6	14	91	364	1001	2002
7	13	78	286	715	1287
8	12	66	220	495	792

- Increase depends on number known and number rotating
- Anything more quickly goes into unreasonable territory for CPU
- Would eventually use 5 fixed, 2 rotating, factor of 105

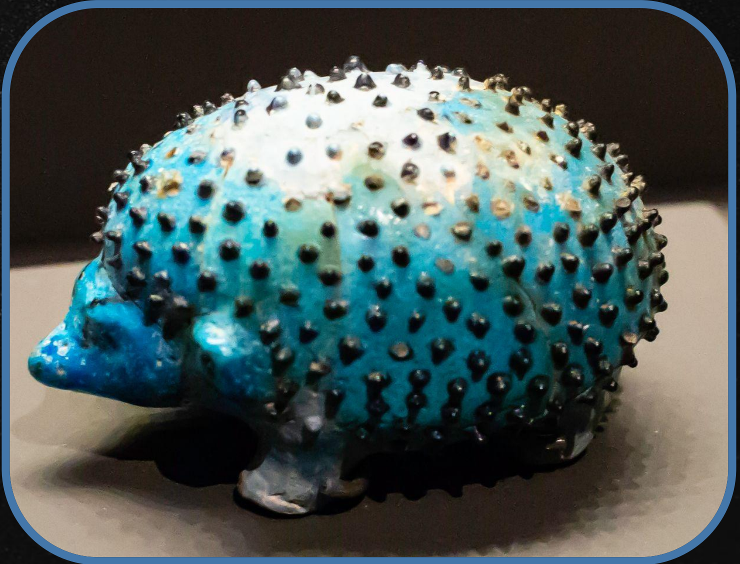
Gotta go fast! Can I use GPUs?

- GPUs are great a “parallel tasks”
  - Doing the same set of operations on different inputs
  - What shaders and renderers need
  - All operations happen simultaneously in lockstep across all inputs
- Often used for other password/crypto cracking
- Also used for crypto mining
- GPU coding is HARD

Non-infringing blue hedgehog. Goes fast.

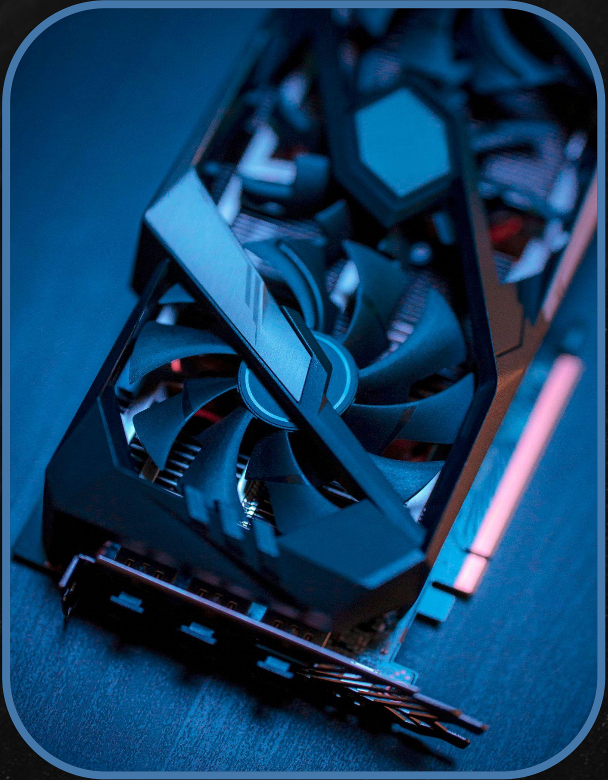
Gotta go fast! Can I use GPUs?

- GPUs are great a “parallel tasks”
  - Doing the same set of operations on different inputs
  - What shaders and renderers need
  - All operations happen simultaneously in lockstep across all inputs
- Often used for other password/crypto cracking
- Also used for crypto mining
- GPU coding is HARD
  - **Has anyone done it for me?**



Non-infringing blue hedgehog. Goes fast.

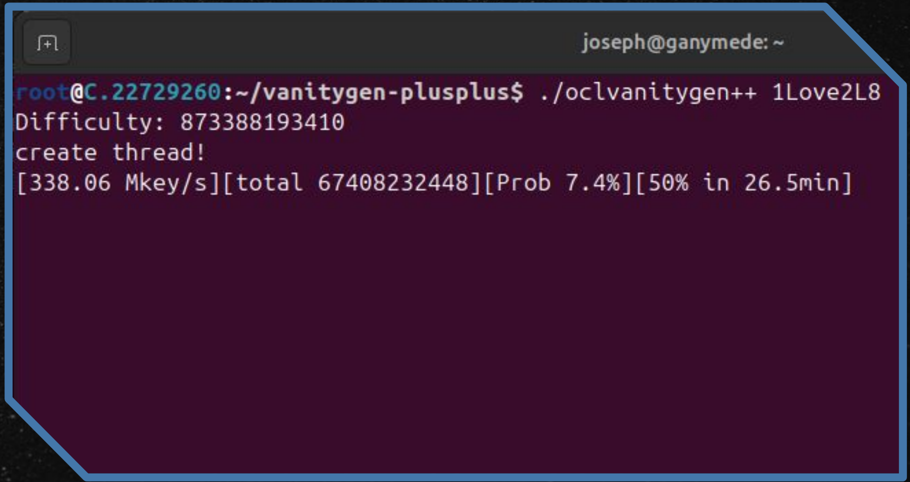
A few GPU-based projects



- vanitygen/oclVanitygen
    - Used for generating “vanity” bitcoin addresses
    - Addresses start with custom string
        - E.g, 1JoeRocksQGefi2DMPTfTL5SLmv7DivfNa
    - oclVanitygen uses GPU
- Bitcrack
    - Designed to solve a different bitcoin “puzzle”
        - Private key is known within a specific range
            - Higher prizes for fewer known bits
        - Focuses more on organized brute forcing
    - Allows specifying a range of private keys to search
- Hashcat
    - General use GPU password cracking
    - Does not support brainwallets (yet!)

Looking at oclVanitygen

- Utility used to generate custom “vanity addresses”
  - github.com/10gic/vanitygen-plusplus
  - Original: github.com/samr7/vanitygen
- Uses GPU to generate millions of addresses
- Keeps going until one matches filter
- Very promising benchmark!
  - ~350 Mkey/s on a 3080
  - This turned out to be a “lie”



```
root@C.22729260:~/vanitygen-plusplus$ ./oclvanitygen++ 1Love2L8
Difficulty: 873388193410
create thread!
[338.06 Mkey/s][total 67408232448][Prob 7.4%][50% in 26.5min]
```

oclVanitygen’s neat little trick

- oclvanitygen doesn’t take in arbitrary private keys
- Can’t be modified to because it uses clever shortcuts for speed
- Uses some clever tricks
  - Batch inversion
  - Mixed-coordinate addition
- Only cares about number of keys tried
  - Just trying to find a pattern match in output
- Uses a random seed, batch inverts

```
* The single most expensive operation performed in this
* loop is modular inversion of ppnt->Z.  There is an
* algorithm implemented in OpenSSL to do batched
* inversion that only does one actual BN_mod_inverse(),
* and saves a _lot_ of time.
*
* To take advantage of this, we batch up a few points,
* and feed them to EC_POINTs_make_affine() below.
*/

        EC_POINTs_make_affine(pgroup, nbatch, ppnt,
        vxcp->vxc_bnctx);

        for (i = 0; i < nbatch; i++, vxcp->vxc_delta++) {
   /* Hash the public key */
   len = EC_POINT_point2oct(pgroup, ppnt[i],
       POINT_CONVERSION_UNCOMPRESSED,
       eckey_buf,
                65,
                vxcp->vxc_bnctx);
   assert(len == 65);

   SHA256(hash_buf, hash_len, hash1);
   RIPEMD160(hash1, sizeof(hash1),
   &vxcp->vxc_binres[1]);

   switch (test_func(vxcp)) {
   …
```

https://github.com/samr7/vanitygen/blob/cd1a7282431dcf7e522777976aa18728ee5bb7be/vanitygen.c#L182

What about Bitcrack?

- Searches for pattern matching a spe
- https://github.com/brichard19/BitCrack
- Also shows very good hashrate
- Allows specifying range of keyspaces
    - This is promising!
- Turns out it’s using the same tricks
- Fool me twice…
    - Can’t get fooled again?

```
__device__ void doIterationWithDouble(int pointsPerThread, int
compression)
{
    unsigned int *chain = _CHAIN[0];
    unsigned int *xPtr = ec::getXPtr();
    unsigned int *yPtr = ec::getYPtr();

    // Multiply together all (_Gx - x) and then invert
    unsigned int inverse[8] = {0,0,0,0,0,0,0,1};
    for(int i = 0; i < pointsPerThread; i++) {
        unsigned int x[8];

        unsigned int digest[5];

        readInt(xPtr, i, x);

        // uncompressed
        if(compression == PointCompressionType::UNCOMPRESSED
|| compression == PointCompressionType::BOTH) {
            unsigned int y[8];
            readInt(yPtr, i, y);
            hashPublicKey(x, y, digest);

            if(checkHash(digest)) {
                setResultFound(i, false, x, y, digest);
            }
        }
```

https://github.com/brichard19/BitCrack/blob/6bf8059ef075eb1622298395866b0bd02375e1d9/CudaKeySearchDevice/CudaKeySearchDevice.cu#L193

What about Hashcat?

- Password cracking utility
  - Open source, widely used
  - ~600 example hashes built-in
  - https://hashcat.net/hashcat/
- GPU accelerated, often optimized
- Has its own generator for wildcard attacks
  - Rather sophisticated customization
- Can also feed in passwords via pipe
- Handles all of the difficult GPU stuff
- Provides framework for custom plugins
- Promising, but no brainwallet support


[redacted]
[redacted]

Is this the hacker known as hashcat?

Admitting defeat. I’m not gonna learn openCL over this.

Alright. Let’s face facts.

- I’ve brute forced all the clues I haven’t solved
- I have high confidence in all my other answers
- I’ve reached the feasibility limit of CPU solving
- I don’t know openCL, and it’s hard to learn
- **Is this even solvable?**
  - Why hasn’t the author yoinked back the money?
  - They presumably know the answers

Admitting defeat. I'm not gonna learn openCL over this.

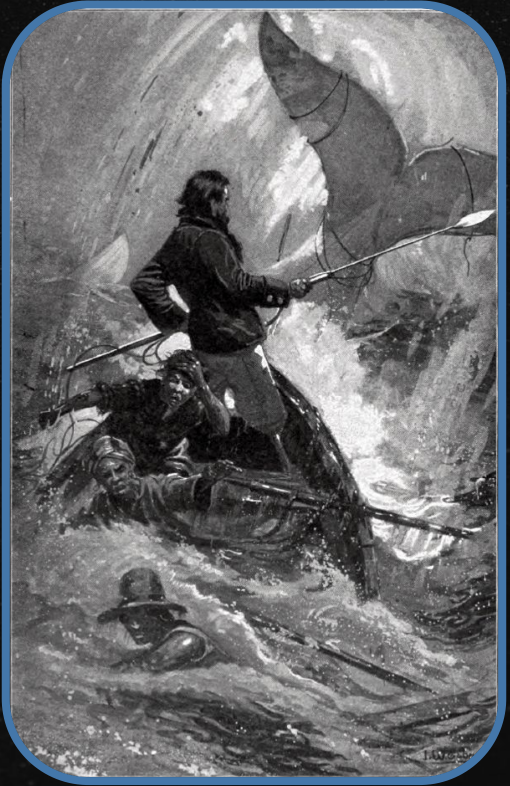
Alright. Let's face facts.

- I've brute forced all the clues I haven't solved
- I have high confidence in all my other answers
- I've reached the feasibility limit of CPU solving
- I don't know openCL, and it's hard to learn
- **Is this even solvable?**
  - Why hasn't the author yoinked back the money?
  - They presumably know the answers
- I'm not like, gonna get weird about this.

Admitting defeat. I'm not gonna learn openCL over this.

Alright. Let's face facts.

- I've brute forced all the clues I haven't solved
- I have high confidence in all my other answers
- I've reached the feasibility limit of CPU solving
- I don't know openCL, and it's hard to learn
- **Is this even solvable?**
    - Why hasn't the author yoinked back the money?
    - They presumably know the answers
- I'm not like, gonna get weird about this.
    - I'm gonna just move on with my life.



Was the real Bitcoin the friends I made along the way?

“It was a good map, and in studying it Mr Wilkinson and his chums would learn a lot about decryption, geography and devious cartography. They wouldn't find in it the whereabouts of AM$150,000 in mixed currencies, though, because the map was a complete and complex fiction.

However, Moist entertained a wonderful warm feeling inside to think that they would, for some time, possess that greatest of all treasures, which is Hope. Anyone who couldn't simply remember where he'd stashed a great big fortune deserved to lose it, in Moist's opinion.”

Terry Pratchett, Going Postal, 2004

GNU Sir Terry Pratchett

On motivation and incentive…

On motivation and incentive…

Screw that: I'll learn openCL

- Hashcat doesn't (didn't!) support brainwallets
- However, Hashcat is modular *and* interactabodular
- Hashcat has GPU-optimized code for all of the operations we need:
  - secp256k1 (the ECC that goes private key -> public key)
  - SHA-256 (used to hash the input, and in address generation)
  - RIPEMD-160 (used in address generation from the public key)
- What if we made them kiss?

Using Hashcat, a Typical Use

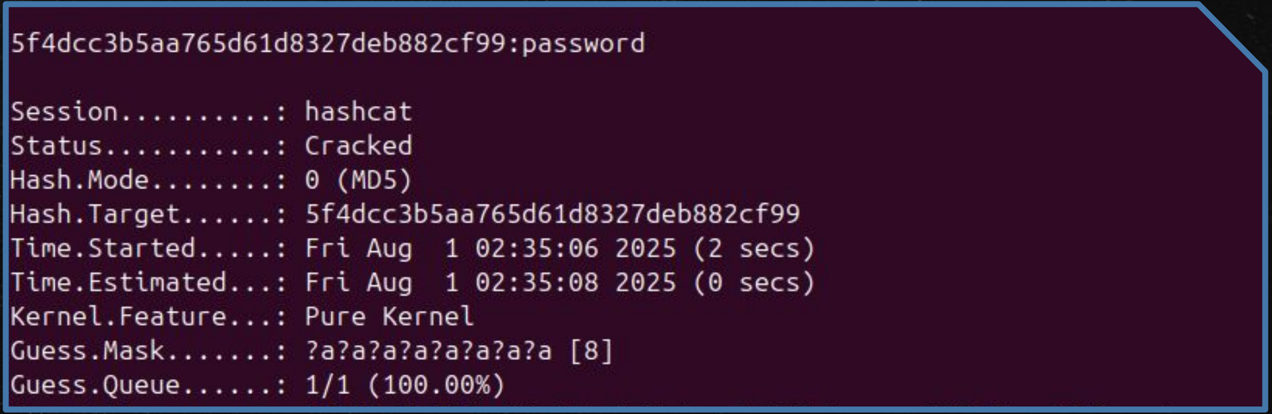
1. Find the hash(es) you want to crack
    a. Figure out what module to use
2. Pick your attack mode
3. Pick a dictionary or generator
    a. This is what gives your "input"
    b. List of common passwords, breach list, etc
    c. Alphanumeric and complex mask patterns
4. Feed it GPUs
5. Check results compulsively every 5 minutes
6. Forget about it for 6 days
7. Check results

Did you know?

90% of Hashcat users quit right before they're about to crack the password. You read it here, it must be true.

Quick Example: MD5

- Hash “password” with MD5: 5f4dcc3b5aa765d61d8327deb882cf99
- Run hashcat in mode 0 (MD5 with no salt), 8 character wildcard mask attack
  - -m 0 -a 3 ?a?a?a?a?a?a?a?a
    - -a 3 sets mask attack mode
    - Each ?a represents a letter a-Z
- 2 seconds later…

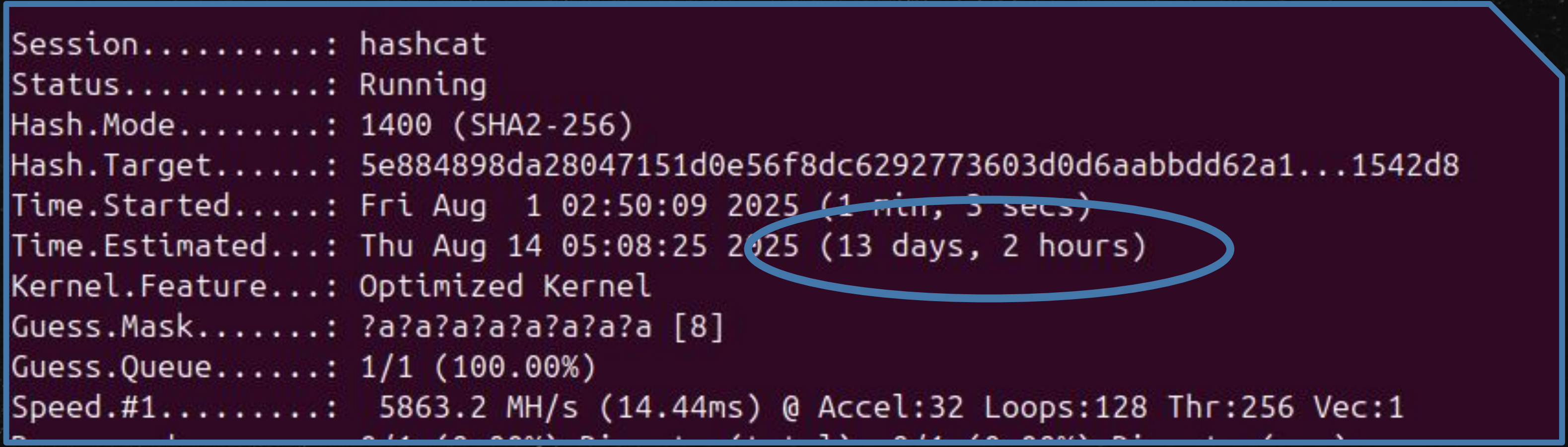


```
5f4dcc3b5aa765d61d8327deb882cf99:password

Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 0 (MD5)
Hash.Target......: 5f4dcc3b5aa765d61d8327deb882cf99
Time.Started.....: Fri Aug  1 02:35:06 2025 (2 secs)
Time.Estimated...: Fri Aug  1 02:35:08 2025 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.......: ?a?a?a?a?a?a?a?a [8]
Guess.Queue......: 1/1 (100.00%)
```

Quick Example: SHA-256

- SHA-256: 5f4dcc3b5aa765d61d8327deb882cf99
- Perform the same attack, this time with mode 1400 (SHA-256, no salt)
  - -m 1400 -a 3 ?a?a?a?a?a?a?a?a
- Estimated to take 13 days!



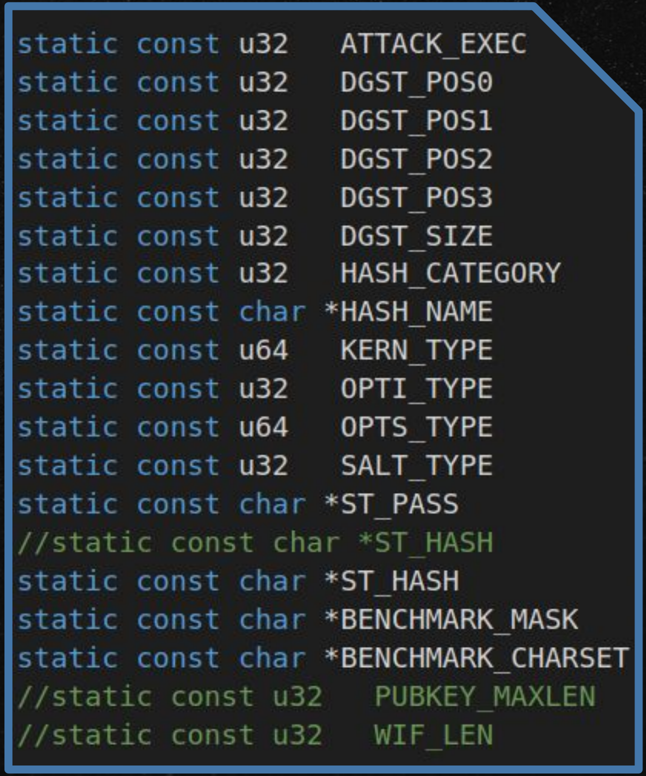
```
Session..........: hashcat
Status...........: Running
Hash.Mode........: 1400 (SHA2-256)
Hash.Target......: 5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a1...1542d8
Time.Started.....: Fri Aug  1 02:50:09 2025 (1 min, 3 secs)
Time.Estimated...: Thu Aug 14 05:08:25 2025 (13 days, 2 hours)
Kernel.Feature...: Optimized Kernel
Guess.Mask.......: ?a?a?a?a?a?a?a?a [8]
Guess.Queue......: 1/1 (100.00%)
Speed.#1.........:  5863.2 MH/s (14.44ms) @ Accel:32 Loops:128 Thr:256 Vec:1
```

What is a plugin?

- Consists of 2 parts:
  - Module
    - Handles setup an overall “structure”
    - Does input/output processing, results handling
    - Run in CPU
    - Written in C
  - Kernel
    - Handles actual crypto operations
    - Run in GPU
    - Written in OpenCL (.cl extension)
- Represents “how to crack” a specific algorithm

What's in a module?

- Location: /src/modules/module_xxxxx.c
- Programmed in C
- Compiled beforehand
  - Either separate, or as part of building Hashcat
- Provides information about the target
  - Type, size, tests, validation parameters, etc
  - Digest information (very important!)
    - Size and ordering segfault if not right
- Provies target-specific functions
  - Encode/decode for input/output formats
  - Over 68 functions/options to implement

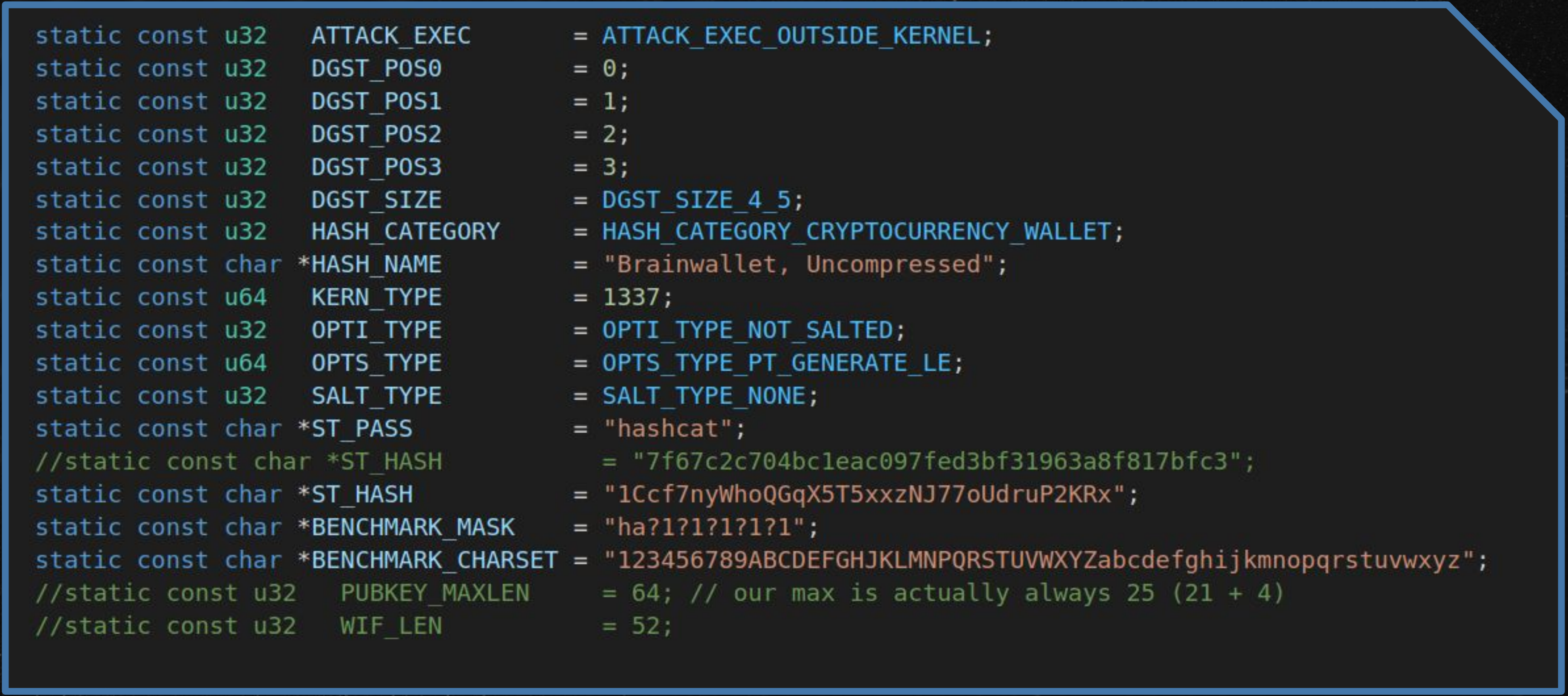


```
static const u32   ATTACK_EXEC
static const u32   DGST_POS0
static const u32   DGST_POS1
static const u32   DGST_POS2
static const u32   DGST_POS3
static const u32   DGST_SIZE
static const u32   HASH_CATEGORY
static const char *HASH_NAME
static const u64   KERN_TYPE
static const u32   OPTI_TYPE
static const u64   OPTS_TYPE
static const u32   SALT_TYPE
static const char *ST_PASS
//static const char *ST_HASH
static const char *ST_HASH
static const char *BENCHMARK_MASK
static const char *BENCHMARK_CHARSET
//static const u32   PUBKEY_MAXLEN
//static const u32   WIF_LEN
```

Parts of a module (some of them, at least)

- `module_hash_decode()`
  - Handles every line in your hash (target) file
  - Turns hash into standardized internal hashcat format
  - Populates hash digests for later comparison
- `module_hash_encode()`
  - Converts internal hashcat format to specific hash format
  - Used to convert cracked hashes for display
- `DGST_SIZE`
  - Specifies the size and number of buffers required for each hash digest
  - Will cause segfaults if you get this wrong (ask me how I know)
- `ST_PASS/ST_HASH`
  - Known password and hash value to use for self-tests
- `BENCHMARK_MASK/BENCHMARK_CHARSET`
  - The mask and allowed set of characters for benchmark mode (**`-b`**)

An example of my module file



```c
static const u32   ATTACK_EXEC       = ATTACK_EXEC_OUTSIDE_KERNEL;
static const u32   DGST_POS0         = 0;
static const u32   DGST_POS1         = 1;
static const u32   DGST_POS2         = 2;
static const u32   DGST_POS3         = 3;
static const u32   DGST_SIZE         = DGST_SIZE_4_5;
static const u32   HASH_CATEGORY     = HASH_CATEGORY_CRYPTOCURRENCY_WALLET;
static const char *HASH_NAME         = "Brainwallet, Uncompressed";
static const u64   KERN_TYPE         = 1337;
static const u32   OPTI_TYPE         = OPTI_TYPE_NOT_SALTED;
static const u64   OPTS_TYPE         = OPTS_TYPE_PT_GENERATE_LE;
static const u32   SALT_TYPE         = SALT_TYPE_NONE;
static const char *ST_PASS           = "hashcat";
//static const char *ST_HASH           = "7f67c2c704bc1eac097fed3bf31963a8f817bfc3";
static const char *ST_HASH           = "1Ccf7nyWhoQGqX5T5xxzNJ77oUdruP2KRx";
static const char *BENCHMARK_MASK    = "ha?1?1?1?1?1";
static const char *BENCHMARK_CHARSET = "123456789ABCDEFGHJKLMNPQRSTUVWXYZabcdefghijkmnopqrstuvwxyz";
//static const u32   PUBKEY_MAXLEN     = 64; // our max is actually always 25 (21 + 4)
//static const u32   WIF_LEN           = 52;
```

What’s in a kernel?

- The actual heavy lifting happens here
- Compiled per-card at runtime
- Located in /openCL/
  - Named mxxxxx-<type>
    - E.g m01300_a3-pure.cl or m10700-optimized.cl
- Some kernels are “pure”, others “optimized”
  - “-pure” is human readable code, “-optimized” takes neat shortcuts
    - Optimized can sometimes be black magic hackery
- The “a” suffix indicates attack mode differences
  - Different approaches for brute force vs dictionary
- Contains 3 key functions
  - init, loop, and comp

Parts of a Kernel

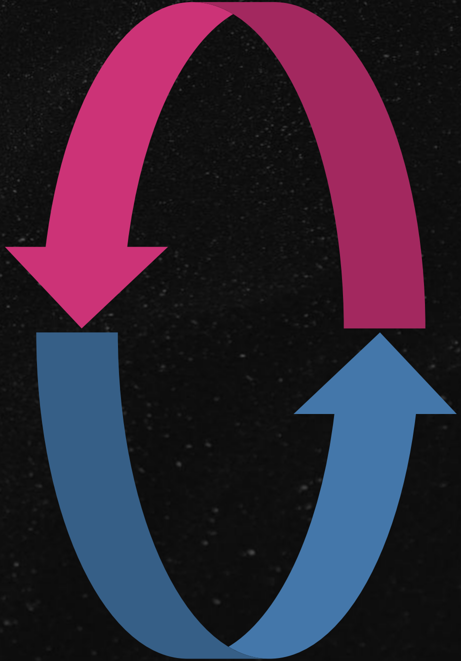
- Init
  - Does the initial setup, initialization, and non-time-intensive calcs
- loop
  - Loop in which time-costly cryptographic operations are completed
- comp
  - Compares the result from each test with the target hashes

The init function

- e.g `KERNEL_FQ void mxxxxx_init (KERN_ATTR_TMPS (brainwallet_tmp_t))`
- Sets up buffers, any one time calculations or initializations
- Run once per candidate
- Sets stuff up for mxxxxx_loop() typically
- In our case we perform the ECC and hashing functions here
    - The last 4 bytes of the result are saved in the digest structure

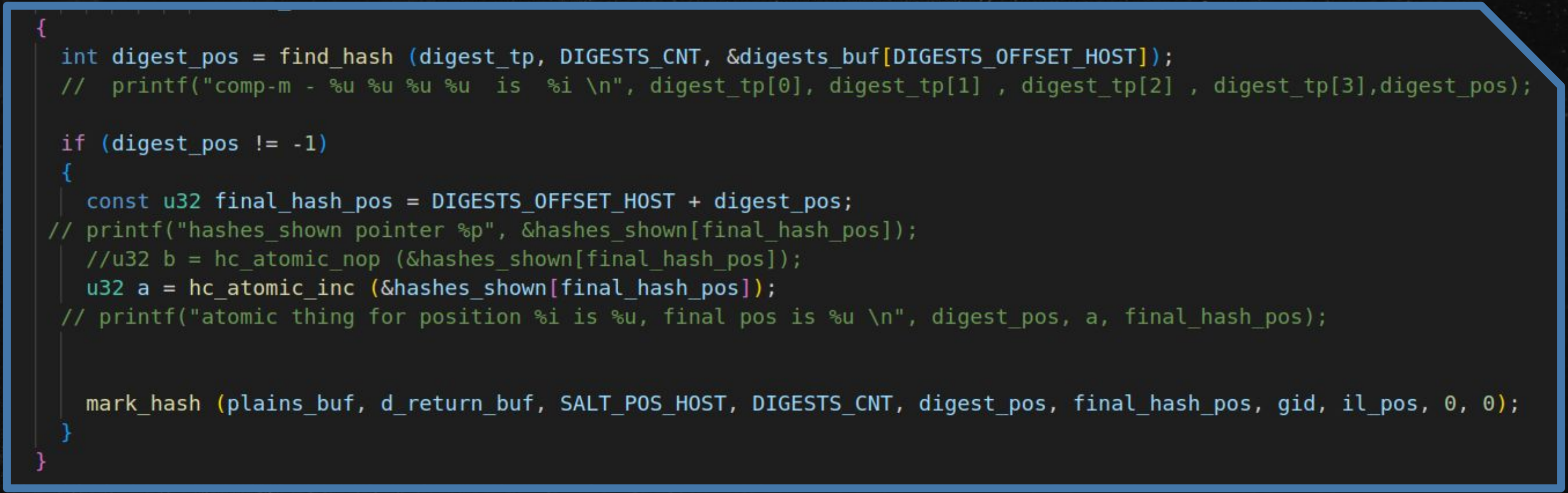
The loop function

- e.g `KERNEL_FQ void mxxxxx_loop (KERN_ATTR_TMPS(brainwallet_tmp_t))`
- Used for crypto algorithms with large rounds of repeated operation
- Helps hashcat break operations down into batches and track progress
- If you’re not doing repeated rounds, everything can happen in `mxxxxx_init()`
- If not skipped, responsible for populating the digest for `mxxxxx_comp()`



The comp function

- e.g `KERNEL_FQ void mxxxxx_comp (KERN_ATTR_TMPS (brainwallet_tmp_t))`
- Compares the result to the list of target results
- Only checks the “digest bits” defined in the module
    - Usually just the ending, but there are edge cases



```
{
  int digest_pos = find_hash (digest_tp, DIGESTS_CNT, &digests_buf[DIGESTS_OFFSET_HOST]);
  //  printf("comp-m - %u %u %u %u  is  %i \n", digest_tp[0], digest_tp[1] , digest_tp[2] , digest_tp[3],digest_pos);

  if (digest_pos != -1)
  {
    const u32 final_hash_pos = DIGESTS_OFFSET_HOST + digest_pos;
 // printf("hashes_shown pointer %p", &hashes_shown[final_hash_pos]);
    //u32 b = hc_atomic_nop (&hashes_shown[final_hash_pos]);
    u32 a = hc_atomic_inc (&hashes_shown[final_hash_pos]);
  // printf("atomic thing for position %i is %u, final pos is %u \n", digest_pos, a, final_hash_pos);


    mark_hash (plains_buf, d_return_buf, SALT_POS_HOST, DIGESTS_CNT, digest_pos, final_hash_pos, gid, il_pos, 0, 0);
  }
}
```

Gathering the Ingredients

- SHA256 (`inc_hash_sha256.cl`)
  - Take functions `sha256_init(), sha256_update(), sha256_final()`
- RIPEMD-160 (`inc_hash_ripemd160.cl`)
  - Take functions `ripemd160_init(), ripemd160_update_swap(), ripemd160_final()`
- secp256k1 (`inc_ecc_secp256k1.cl`)
  - Take functions `point_mul_xy(), set_precomputed_basepoint_g()`
- Worked off of module for secpk256k1 as template
  - `module_28502.c` handles “Bitcoin WIF private key (P2PKH), uncompressed”
  - Use to guide output and memory requirements
- Modified data input based off module for SHA256
  - `module_01400.c` handles SHA256
  - Use to guide input handling and processing

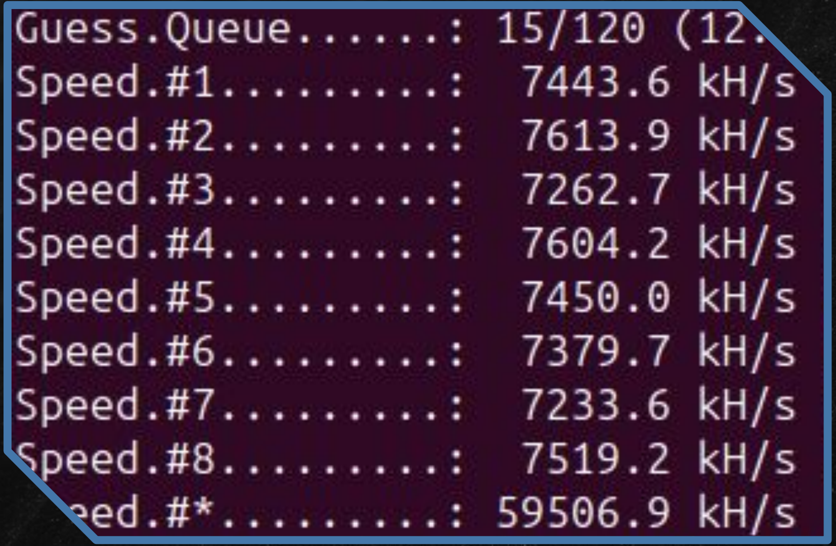
Changes to the module

- Take the input parameters
- Take decode function from target (module 28502)
- Take encode function from

Testing, debugging, and your new best friend printf()

- Debugging openCL code is hard
- You can’t use traditional debuggers in GPU
- printf() is somehow the best (and recommended) way
- You get one console message per worker thread… so it’s spammy
    - Recommendation is to use specific flags to force a single thread for debugging
        - `-n 1 -u 1 -T 1`
- If things aren’t working, check your digest sizes
- If things run but the results are wrong, check your endianness

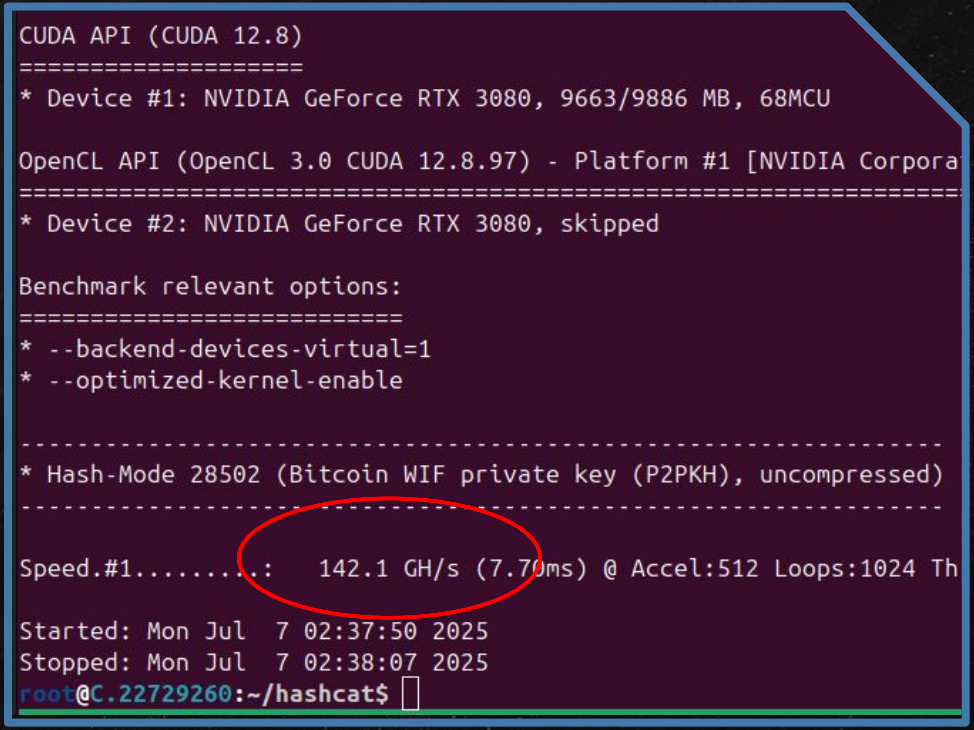
It works! But slowly...?



```
Guess.Queue.......: 15/120 (12
Speed.#1.........:  7443.6 kH/s
Speed.#2.........:  7613.9 kH/s
Speed.#3.........:  7262.7 kH/s
Speed.#4.........:  7604.2 kH/s
Speed.#5.........:  7450.0 kH/s
Speed.#6.........:  7379.7 kH/s
Speed.#7.........:  7233.6 kH/s
Speed.#8.........:  7519.2 kH/s
 eed.#*.........: 59506.9 kH/s
```

- Initial benchmark was disappointing
- Why were the individual modules so much faster?
- Did I add a slowdown?
- Is it damn batch inversion again?
  - If it is I’m going to invert my computer desk
- I’m getting around 7.5 MHash
  - Benchmark for individual components was much higher

Say it with me: “The Benchmark Was a Lie”



```
CUDA API (CUDA 12.8)
====================
* Device #1: NVIDIA GeForce RTX 3080, 9663/9886 MB, 68MCU

OpenCL API (OpenCL 3.0 CUDA 12.8.97) - Platform #1 [NVIDIA Corpora
===================================================================
* Device #2: NVIDIA GeForce RTX 3080, skipped

Benchmark relevant options:
===========================
* --backend-devices-virtual=1
* --optimized-kernel-enable

-----------------------------------------------------------------
* Hash-Mode 28502 (Bitcoin WIF private key (P2PKH), uncompressed)
-----------------------------------------------------------------

Speed.#1.........:   142.1 GH/s (7.70ms) @ Accel:512 Loops:1024 Th

Started: Mon Jul  7 02:37:50 2025
Stopped: Mon Jul  7 02:38:07 2025
root@C.22729260:~/hashcat$
```

- Individual modules had insane hashrate, mine did not.
- This time it’s not batch inversion!
- Hashcat used a lot of early filters to filter out non-valid private keys before doing any calculations
- It’s a lot slower when you’re only passing in valid keys

Generator Generator

- Hashcat can accept a list of masks to use
- I created a small python script to generate a mask for each possible mask
- These masks are used to generate all the possible candidates

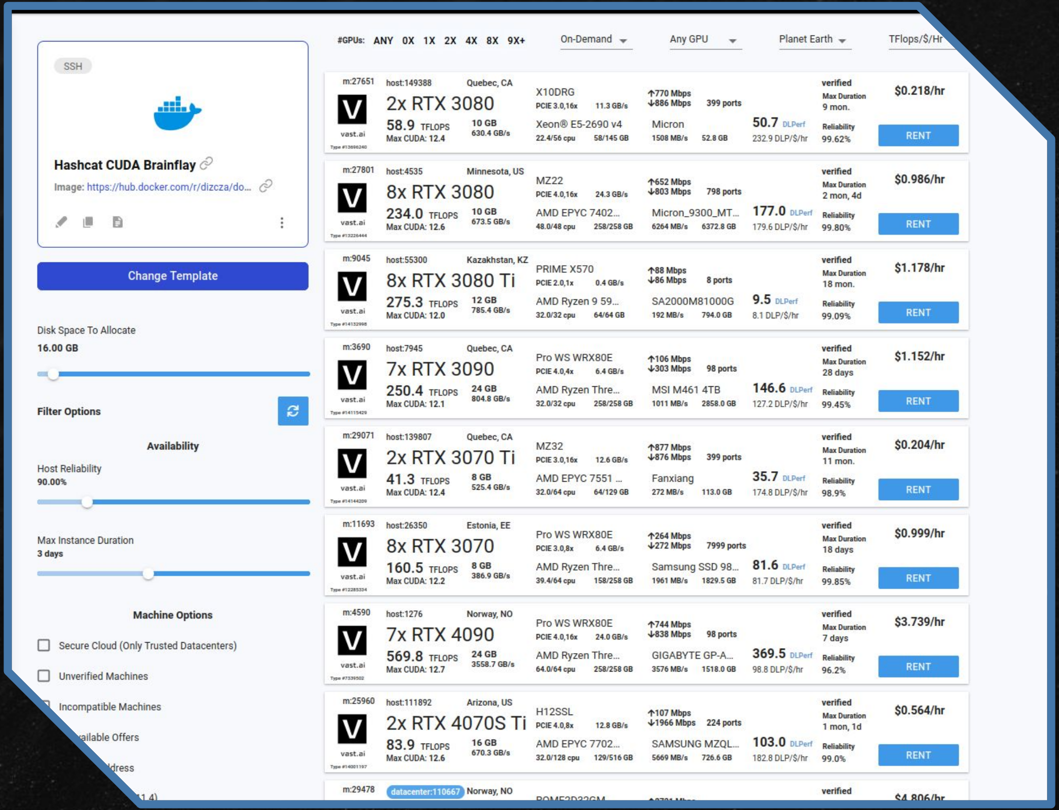
Cracking the puzzle, for real.

I have everything I need:

- Hashcat plugin for Brainwallet
- A generator to make the list of masks
- A new disdain for openCL

Let’s do it!

To the Cloud!



SSH

Hashcat CUDA Brainflay

Image: https://hub.docker.com/r/dizcza/do…

Change Template

Disk Space To Allocate
16.00 GB

Filter Options

Availability

Host Reliability
90.00%

Max Instance Duration
3 days

Machine Options

☐ Secure Cloud (Only Trusted Datacenters)
☐ Unverified Machines
☐ Incompatible Machines
ailable Offers
dress
1.4)

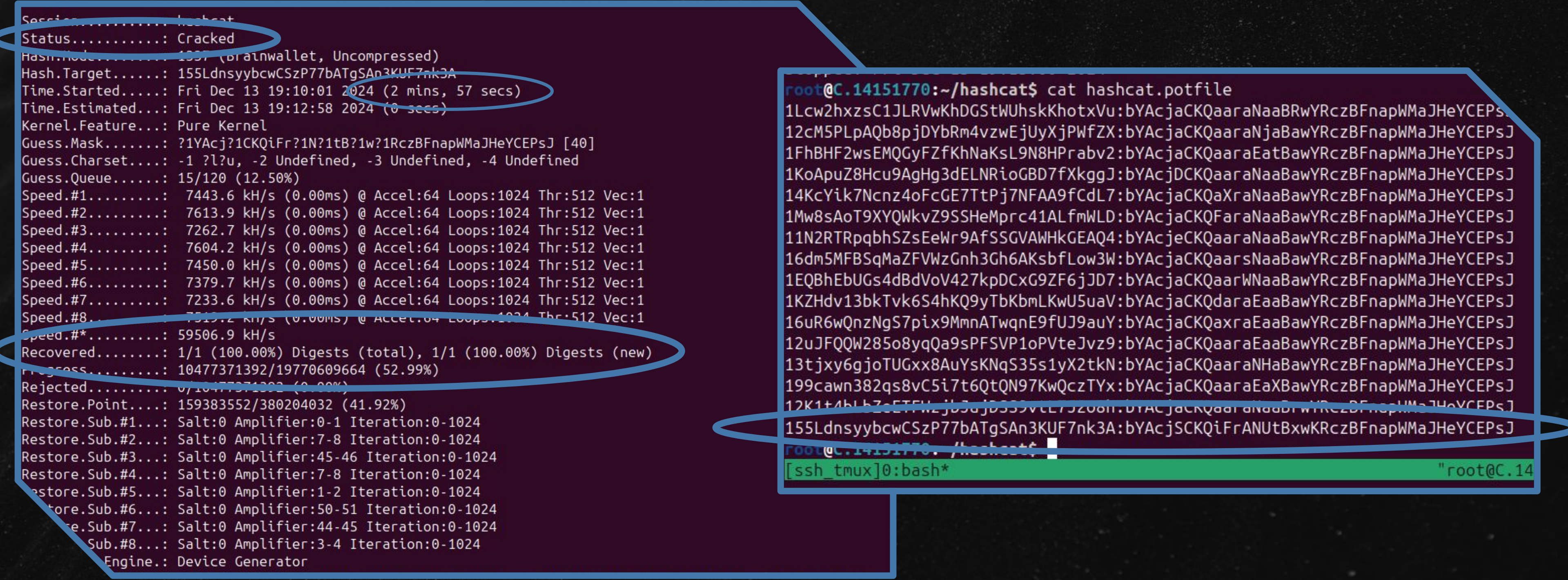
#GPUs: ANY 0X 1X 2X 4X 8X 9X+ On-Demand Any GPU Planet Earth TFlops/$/Hr

m:27651 vast.ai	host:149388	Quebec, CA	2x RTX 3080	58.9 TFLOPS Max CUDA: 12.4	10 GB 630.4 GB/s	X10DRG PCIE 3.0,16x 11.3 GB/s	Xeon® E5-2690 v4 22.4/56 cpu 58/145 GB	↑770 Mbps ↓886 Mbps	399 ports	Micron 1508 MB/s 52.8 GB	50.7 DLPerf 232.9 DLP/$/hr	verified Max Duration 9 mon. Reliability 99.62%	$0.218/hr RENT
m:27801 vast.ai	host:4535	Minnesota, US	8x RTX 3080	234.0 TFLOPS Max CUDA: 12.6	10 GB 673.5 GB/s	MZ22 PCIE 4.0,16x 24.3 GB/s	AMD EPYC 7402… 48.0/48 cpu 258/258 GB	↑652 Mbps ↓803 Mbps	798 ports	Micron_9300_MT… 6264 MB/s 6372.8 GB	177.0 DLPerf 179.6 DLP/$/hr	verified Max Duration 2 mon, 4d Reliability 99.80%	$0.986/hr RENT
m:9045 vast.ai	host:55300	Kazakhstan, KZ	8x RTX 3080 Ti	275.3 TFLOPS Max CUDA: 12.0	12 GB 785.4 GB/s	PRIME X570 PCIE 2.0,1x 0.4 GB/s	AMD Ryzen 9 59… 32.0/32 cpu 64/64 GB	↑88 Mbps ↓86 Mbps	8 ports	SA2000M81000G 192 MB/s 794.0 GB	9.5 DLPerf 8.1 DLP/$/hr	verified Max Duration 18 mon. Reliability 99.09%	$1.178/hr RENT
m:3690 vast.ai	host:7945	Quebec, CA	7x RTX 3090	250.4 TFLOPS Max CUDA: 12.1	24 GB 804.8 GB/s	Pro WS WRX80E PCIE 4.0,4x 6.4 GB/s	AMD Ryzen Thre… 32.0/32 cpu 258/258 GB	↑106 Mbps ↓303 Mbps	98 ports	MSI M461 4TB 1011 MB/s 2858.0 GB	146.6 DLPerf 127.2 DLP/$/hr	verified Max Duration 28 days Reliability 99.45%	$1.152/hr RENT
m:29071 vast.ai	host:139807	Quebec, CA	2x RTX 3070 Ti	41.3 TFLOPS Max CUDA: 12.4	8 GB 525.4 GB/s	MZ32 PCIE 3.0,16x 12.6 GB/s	AMD EPYC 7551 … 32.0/64 cpu 64/129 GB	↑877 Mbps ↓876 Mbps	399 ports	Fanxiang 272 MB/s 113.0 GB	35.7 DLPerf 174.8 DLP/$/hr	verified Max Duration 11 mon. Reliability 98.9%	$0.204/hr RENT
m:11693 vast.ai	host:26350	Estonia, EE	8x RTX 3070	160.5 TFLOPS Max CUDA: 12.2	8 GB 386.9 GB/s	Pro WS WRX80E PCIE 3.0,8x 6.4 GB/s	AMD Ryzen Thre… 39.4/64 cpu 158/258 GB	↑264 Mbps ↓272 Mbps	7999 ports	Samsung SSD 98… 1961 MB/s 1829.5 GB	81.6 DLPerf 81.7 DLP/$/hr	verified Max Duration 18 days Reliability 99.85%	$0.999/hr RENT
m:4590 vast.ai	host:1276	Norway, NO	7x RTX 4090	569.8 TFLOPS Max CUDA: 12.7	24 GB 3558.7 GB/s	Pro WS WRX80E PCIE 4.0,16x 24.0 GB/s	AMD Ryzen Thre… 64.0/64 cpu 258/258 GB	↑744 Mbps ↓838 Mbps	98 ports	GIGABYTE GP-A… 3576 MB/s 1518.0 GB	369.5 DLPerf 98.8 DLP/$/hr	verified Max Duration 7 days Reliability 96.2%	$3.739/hr RENT
m:25960 vast.ai	host:111892	Arizona, US	2x RTX 4070S Ti	83.9 TFLOPS Max CUDA: 12.6	16 GB 670.3 GB/s	H12SSL PCIE 4.0,8x 12.8 GB/s	AMD EPYC 7702… 32.0/128 cpu 129/516 GB	↑107 Mbps ↓1966 Mbps	224 ports	SAMSUNG MZQL… 5669 MB/s 726.6 GB	103.0 DLPerf 182.8 DLP/$/hr	verified Max Duration 1 mon, 1d Reliability 99.0%	$0.564/hr RENT
m:29478	datacenter:110667	Norway, NO										verified	$4.806/hr

- vast.ai is a cloud GPU provider
- Relatively cheap
- Already has a hashcat docker image
- Script to pull down and compile custom code
- Way cheaper than buying cards for a cracking rig

SEVERAL HOURS LATER

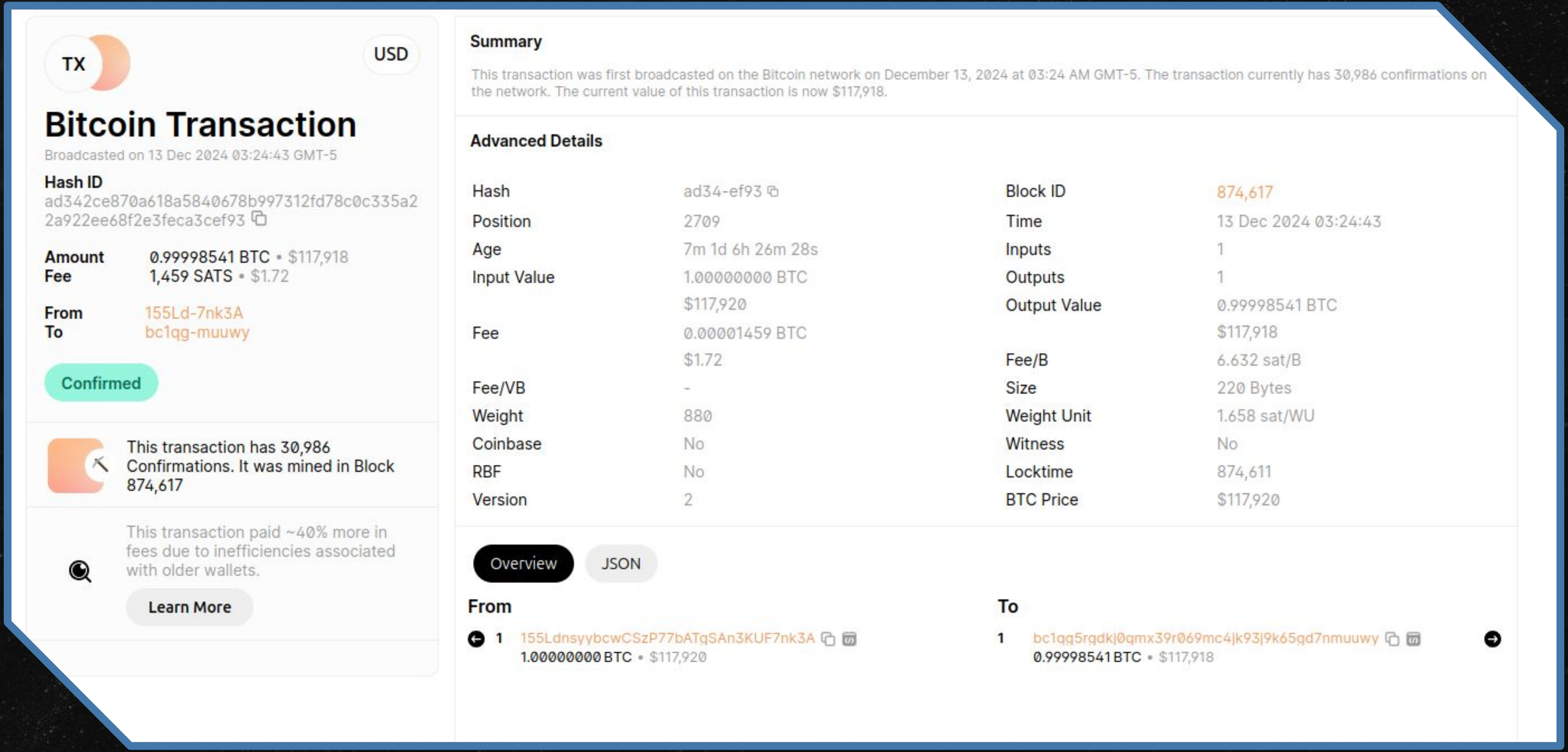
Success! In under 3 minutes, too!



```
Session..........: hashcat
Status...........: Cracked
Hash.Mode........: 1337 (Brainwallet, Uncompressed)
Hash.Target......: 155LdnsyybcwCSzP77bATgSAn3KUF7nk3A
Time.Started.....: Fri Dec 13 19:10:01 2024 (2 mins, 57 secs)
Time.Estimated...: Fri Dec 13 19:12:58 2024 (0 secs)
Kernel.Feature...: Pure Kernel
Guess.Mask.......: ?1YAcj?1CKQiFr?1N?1tB?1w?1RczBFnapWMaJHeYCEPsJ [40]
Guess.Charset....: -1 ?l?u, -2 Undefined, -3 Undefined, -4 Undefined
Guess.Queue......: 15/120 (12.50%)
Speed.#1.........:  7443.6 kH/s (0.00ms) @ Accel:64 Loops:1024 Thr:512 Vec:1
Speed.#2.........:  7613.9 kH/s (0.00ms) @ Accel:64 Loops:1024 Thr:512 Vec:1
Speed.#3.........:  7262.7 kH/s (0.00ms) @ Accel:64 Loops:1024 Thr:512 Vec:1
Speed.#4.........:  7604.2 kH/s (0.00ms) @ Accel:64 Loops:1024 Thr:512 Vec:1
Speed.#5.........:  7450.0 kH/s (0.00ms) @ Accel:64 Loops:1024 Thr:512 Vec:1
Speed.#6.........:  7379.7 kH/s (0.00ms) @ Accel:64 Loops:1024 Thr:512 Vec:1
Speed.#7.........:  7233.6 kH/s (0.00ms) @ Accel:64 Loops:1024 Thr:512 Vec:1
Speed.#8.........:  7519.2 kH/s (0.00ms) @ Accel:64 Loops:1024 Thr:512 Vec:1
Speed.#*.........: 59506.9 kH/s
Recovered........: 1/1 (100.00%) Digests (total), 1/1 (100.00%) Digests (new)
Progress.........: 10477371392/19770609664 (52.99%)
Rejected.........: 0/10477371392 (0.00%)
Restore.Point....: 159383552/380204032 (41.92%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1024
Restore.Sub.#2...: Salt:0 Amplifier:7-8 Iteration:0-1024
Restore.Sub.#3...: Salt:0 Amplifier:45-46 Iteration:0-1024
Restore.Sub.#4...: Salt:0 Amplifier:7-8 Iteration:0-1024
estore.Sub.#5...: Salt:0 Amplifier:1-2 Iteration:0-1024
 tore.Sub.#6...: Salt:0 Amplifier:50-51 Iteration:0-1024
  e.Sub.#7...: Salt:0 Amplifier:44-45 Iteration:0-1024
    Sub.#8...: Salt:0 Amplifier:3-4 Iteration:0-1024
      Engine.: Device Generator
```

```
root@C.14151770:~/hashcat$ cat hashcat.potfile
1Lcw2hxzsC1JLRVwKhDGStWUhskKhotxVu:bYAcjaCKQaaraNaaBRwYRczBFnapWMaJHeYCEPs
12cM5PLpAQb8pjDYbRm4vzwEjUyXjPWfZX:bYAcjaCKQaaraNjaBawYRczBFnapWMaJHeYCEPsJ
1FhBHF2wsEMQGyFZfKhNaKsL9N8HPrabv2:bYAcjaCKQaaraEatBawYRczBFnapWMaJHeYCEPsJ
1KoApuZ8Hcu9AgHg3dELNRioGBD7fXkggJ:bYAcjDCKQaaraNaaBawYRczBFnapWMaJHeYCEPsJ
14KcYik7Ncnz4oFcGE7TtPj7NFAA9fCdL7:bYAcjaCKQaXraNaaBawYRczBFnapWMaJHeYCEPsJ
1Mw8sAoT9XYQWkvZ9SSHeMprc41ALfmWLD:bYAcjaCKQFaraNaaBawYRczBFnapWMaJHeYCEPsJ
11N2RTRpqbhSZsEeWr9AfSSGVAWHkGEAQ4:bYAcjeCKQaaraNaaBawYRczBFnapWMaJHeYCEPsJ
16dm5MFBSqMaZFVWzGnh3Gh6AKsbfLow3W:bYAcjaCKQaarsNaaBawYRczBFnapWMaJHeYCEPsJ
1EQBhEbUGs4dBdVoV427kpDCxG9ZF6jJD7:bYAcjaCKQaarWNaaBawYRczBFnapWMaJHeYCEPsJ
1KZHdv13bkTvk6S4hKQ9yTbKbmLKwU5uaV:bYAcjaCKQdaraEaaBawYRczBFnapWMaJHeYCEPsJ
16uR6wQnzNgS7pix9MmnATwqnE9fUJ9auY:bYAcjaCKQaxraEaaBawYRczBFnapWMaJHeYCEPsJ
12uJFQQW285o8yqQa9sPFSVP1oPVteJvz9:bYAcjaCKQaaraEaaBawYRczBFnapWMaJHeYCEPsJ
13tjxy6gjoTUGxx8AuYsKNqS35s1yX2tkN:bYAcjaCKQaaraNHaBawYRczBFnapWMaJHeYCEPsJ
199cawn382qs8vC5i7t6QtQN97KwQczTYx:bYAcjaCKQaaraEaXBawYRczBFnapWMaJHeYCEPsJ
12K1t4bLhZ[illegible]YRczBFnapWMaJHeYCEPsJ
155LdnsyybcwCSzP77bATgSAn3KUF7nk3A:bYAcjSCKQiFrANUtBxwKRczBFnapWMaJHeYCEPsJ
root@C.14151770:~/hashcat$
[ssh_tmux]0:bash*                                                 "root@C.14
```

Claiming the prize



TX

USD

Bitcoin Transaction

Broadcasted on 13 Dec 2024 03:24:43 GMT-5

Hash ID
ad342ce870a618a5840678b997312fd78c0c335a22a922ee68f2e3feca3cef93

Amount 0.99998541 BTC • $117,918
Fee 1,459 SATS • $1.72

From 155Ld-7nk3A
To bc1qg-muuwy

Confirmed

This transaction has 30,986 Confirmations. It was mined in Block 874,617

This transaction paid ~40% more in fees due to inefficiencies associated with older wallets.

Learn More

Summary

This transaction was first broadcasted on the Bitcoin network on December 13, 2024 at 03:24 AM GMT-5. The transaction currently has 30,986 confirmations on the network. The current value of this transaction is now $117,918.

Advanced Details

Hash	ad34-ef93	Block ID	874,617
Position	2709	Time	13 Dec 2024 03:24:43
Age	7m 1d 6h 26m 28s	Inputs	1
Input Value	1.00000000 BTC	Outputs	1
	$117,920	Output Value	0.99998541 BTC
Fee	0.00001459 BTC		$117,918
	$1.72	Fee/B	6.632 sat/B
Fee/VB	-	Size	220 Bytes
Weight	880	Weight Unit	1.658 sat/WU
Coinbase	No	Witness	No
RBF	No	Locktime	874,611
Version	2	BTC Price	$117,920

Overview JSON

From
1 155LdnsyybcwCSzP77bATgSAn3KUF7nk3A
1.00000000 BTC • $117,920

To
1 bc1qg5rgdkj0qmx39r069mc4jk93j9k65gd7nmuuwy
0.99998541 BTC • $117,918

What about the money?

- It’s mostly been spent.
  - Not saying on what.
- I paid my taxes.
  - (even the joker doesn't mess with the IRS)
  - Getting an accountant to call me back was hard
- The rest is not on my person, in my home, or on any online wallet/account.
  - Or a brainwallet, at that. Please don’t rob me.
- I only did one outrageous thing...

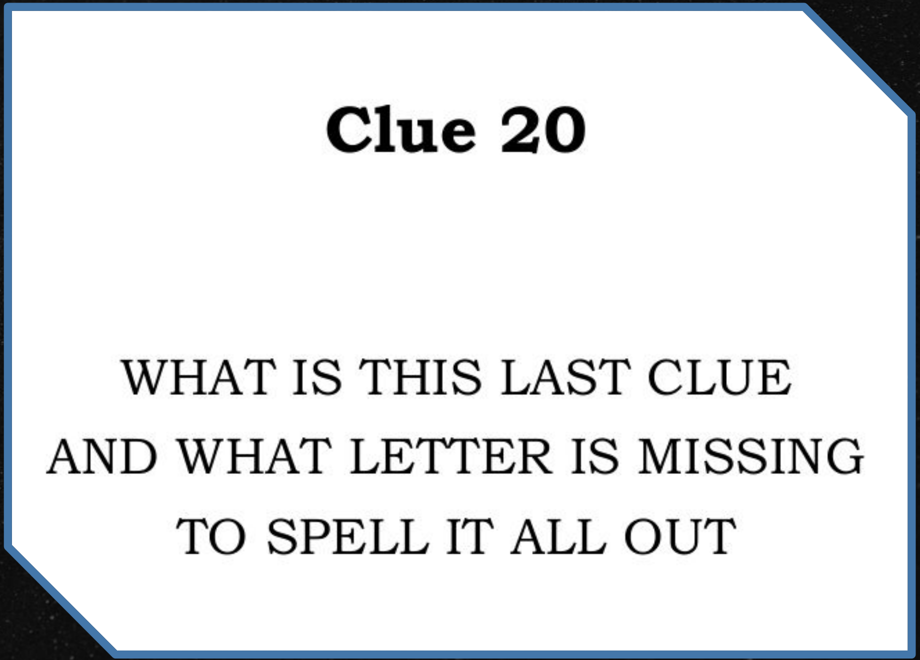
What about the money?

- It’s mostly been spent.
  - Not saying on what.
- I paid my taxes.
  - (even the joker doesn't mess with the IRS)
  - Getting an accountant to call me back was hard
- The rest is not on my person, in my home, or on any online wallet/account.
  - Or a brainwallet, at that. Please don’t rob me.
- I only did one outrageous thing…
  - **ZERO GRAVITY FLIGHT!!!**

Photo Credit: Zero-Gravity Corporation, Paul Gramaglia and Stephen Boxall

The Frustrating Thing About Clue 20

- This is the clue that prevented the earlier tries from working
- Ambiguous solution
- Clue is “TWENTY”, letter “Y” does not appear
- It’s also a Haiku (5-7-5)
- Letter “K” also doesn’t appear
- It was K. Womp womp.



Clue 20

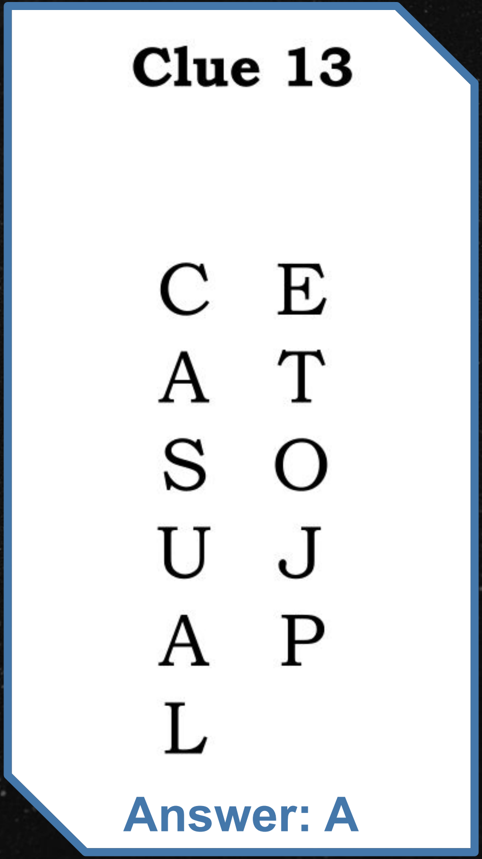
WHAT IS THIS LAST CLUE
AND WHAT LETTER IS MISSING
TO SPELL IT ALL OUT

Unsolved Clues - 13 and 18

- I know the answers, but not the solution. It's killing me inside.
- I am offering a small trophy to whoever can give me a satisfying explanation.
- Please give my soul peace.
- Email answers to:

puzzleanswers@begaydocrime.com

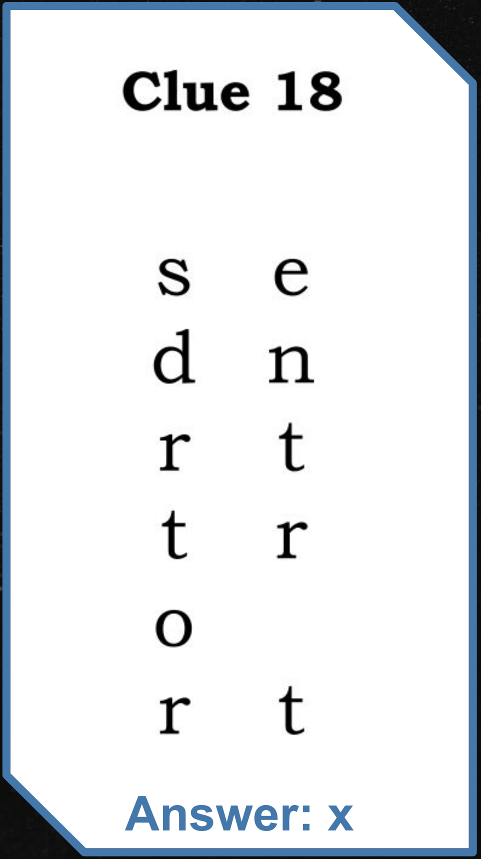
Or, write them on the back of a $20 bill and mail them to my PO Box.



Clue 13

C E
A T
S O
U J
A P
L

Answer: A



Clue 18

s e
d n
r t
t r
o
r t

Answer: x

Trying it at Home

Custom module is available on my github: github.com/stoppingcart/puzzlecat

Usage instructions (remember, you get what you pay for):

- Download Hashcat source
- Download my files
- Add custom modules to the Hashcat modules folder
- Do the same for the kernels
- Build hashcat as normal, install dependencies, cry, etc. etc.
- Run hashcat in mode -m 1337 or 1338
    - Uncompressed vs. compressed mode respectively (affects public key -> address step)

Acknowledgements - Huge thanks to the following:

- **The Hashcat Team** for making and open sourcing such great software and fantastic documentation
- **Ryan Castellucci** for creating Brainflayer and fighting the good fight
- **Spencer Lucas**, whoever you are, for creating this fun challenge
- **My parents (hi mom!), my friends, and everyone** who supported me along the way.
- **My partner Daisy** for encouraging me even when I was frustrated
- **My accountant Jill**, for actually returning my call and hearing me out
- **Chuck Copenspire**, for help and advice on my powerpoint design
  - Creative and technical consultant for queer + neurodivergent professionals
  - See: chuckcopenspire.com
- **My friend Ashley** for keeping this fixation alive
- The folks at Zero G for an unforgettable experience

Please donate to Ryan, they rock.

Legalise Nonbinary UK - Help Cover Ryan's Legal Costs

£6,996 raised
£30K goal · 134 donations

Ryan Castellucci and Emma Drury are organizing this fundraiser.

https://www.gofundme.com/f/legalise-nonbinary-uk-help-cover-ryans-legal-costs

That’s all folks,
Thank you all for coming!

I’ll be answering questions I know the answer to, and telling lies about the ones I don’t.